МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

СОШИАЛ МОНИТОРИНГИЙН СИСТЕМ ДЭХ АГУУЛГЫН ШИНЖИЛГЭЭ, СЭДЭВ ЗАГВАРЧЛАЛЫН ШИЙДЭЛ

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)
Бакалаварын судалгааны ажил

Удирдагч: Ж.Үйтүмэн

Гүйцэтгэс эн: П.Номин-Эрдэнэ (22b1num4640)

Улаанбаатар
2026 оны 5 сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Пүрэвдоржийн Номин-Эрдэнэ

СОШИАЛ МОНИТОРИНГИЙН СИСТЕМ ДЭХ АГУУЛГЫН ШИНЖИЛГЭЭ, СЭДЭВ ЗАГВАРЧЛАЛЫН ШИЙДЭЛ

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)

Бакалаварын судалгааны ажил

Улаанбаатар
2026 оны 5 сар

Зохиогчийн баталгаа

Миний бие Пүрэвдоржийн Номин-Эрдэнэ ”Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл” сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ эсвэл ихэнхдээ Монгол Улсын Их Сургуулийн зэрэг горилохоор дэвшүүлсэн болно.
- Энэ ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй.
- Бусдын хийсэн ажлаас хуулбарлаагүй, ашигласан бол ишлэл, зүүлт хийсэн.
- Ажлыг би өөрөө (хамтарч) хийсэн ба миний хийсэн ажил, үзүүлсэн дэмжлэгийг дипломын ажилд тодорхой тусгасан.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____

Огноо: 2021.05.08.

Программ Хангамжийн Инженерийн ангийн оюутан П.Номин-Эрдэнийн “Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл” сэдэвт бакалаврын дипломын ажлын шүүмж

Энэхүү ажил нь сошиал медиа дахь том хэмжээний монгол хэл дээрх өгөгдөлд задлан шинжилгээ хийх, үндсэн сэдвийг тодорхойлох болон нэрлэсэн нэгжийг таних үйлдэл бүхий системийг хөгжүүлсэн байна. Үүгээр цахим орчин дахь тренд болон олон нийтийн санаа бодлыг шуурхай хянах боломжийг байгууллагуудад олгоно.

Дараах ажлыг хийж гүйцэтгэсэн байна. Үүнд:

- *NLP дамжлага:* Текст боловсруулах бүрэн дамжлагыг (Боловсруулалт --> Шинжилгээ -> Гаралт буюу үр дүн) хэрэгжүүлсэн. Монгол хэлний онцлогт тохирсон боловсруулалт хийсэн нь маш сайн болсон.
- *Техникийн шийдэл:* FastAPI ашиглан асинхрон байдлаар өгөгдлийг боловсруулсан нь системийн хурдыг нэмэгдүүлсэн. NEREngine болон TopicModeler гэх мэт модульд хуваасан нь кодын зохион байгуулалт сайтайг илтгэнэ.
- *Гүйцэтгэл:* Том хэмжээний өгөгдлийг "Багц" байдлаар боловсруулж байгаа нь практик хэрэглээний шаардлагыг хангаж байна.

Энэ ажлын давуу талыг дурдвал:

- *Хэлний онцлогийг тусгасан:* Монгол хэлний зүй тогтлыг NLP-ийн өмнөх шатанд зөв тооцсон.
- *Иж бүрэн шийдэл:* Зөвхөн алгоритм төдийгүй, түүнийг ашиглах вэб үйлчилгээний (API) хэсгийг бүрэн боловсруулж, турших веб программ хөгжүүлсэн.

П.Номин-Эрдэнийн гүйцэтгэсэн ажил нь өгөгдлийн шинжилгээ болон програм хангамжийн инженерчлэлийн орчин үеийн аргыг амжилттай хослуулсан, практик ач холбогдол өндөртэй шилдэг ажил болжээ. Тиймээс 5 онооноос 4.8 оноо өгөх саналтай байна.

МКУТ-ийн ахлах багш Ph.D. Б.Батням /

МУИС-ийн Мэдээллийн Технологи, Электроникийн Сургууль

Бакалаврын төгсөлтийн ажлын удирдагчийн тодорхойлолт

2026 оны 05 дугаар сарын 07

Тэнхим: Мэдээлэл, компьютерийн ухааны тэнхим
Горилогч: П.Номин-Эрдэнэ
Судалгааны ажлын сэдэв: Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл

Оюутан Номин-Эрдэнийн сэдэв нь нийгмийн сүлжээн дэх агуулгад шинжилгээ хийх аргад дэвшилтэт аргачлал нэвтрүүлэхэд чиглэсэн бөгөөд сэдвийн болон Монгол хэлний онцлог нь энэхүү ажлыг гүйцэтгэхэд онолын гүнзгий мэдлэг, ихээхэн судалгаа, туршилт, програмчлалын олон талт мэдлэг, технологи шаардсан ажил болсон.

Сэдвийн хүрээнд Монгол хэл дээр агуулгын шинжилгээг хийхэд эхний ээлжинд оноосон нэр, сэдвийг ялгаж таних боломжийг бий болгох хэрэгтэй байсан. Энэхүү ажлын үр дүнд агуулгын сэдвүүдийг урьдчилан хүн оноох ажиллагааг халж автоматаар сэдвийг илрүүлэх, нийлмэл нэрүүдийг үг тус бүрээр нь ялгаж буруу ангилахгүй болгож, нэрэнд байдаг дундуур зураас, цэг зэргийг нэг нэршил болгон таних зэрэг олон асуудлуудыг шийдсэн ба өөрчлөлт оруулахад хялбар, олон өөр системд ашиглах, интегрэйшн хийх боломжтой зургаан талт архитектураар хэрэгжүүсэн нь бодит хэрэглээнд нэвтрүүлэх боломжтой болсон.

Хэдийгээр өгөгдлөө цуглуулах, өгөгдлийн хэмжээнээс хамааран зарим алгортмууд хангалтгүй гүйцэтгэл үзүүлэх, гараар тэмдэглэх зэрэг цаг хугацаа болон нөөцийн хязгаарлалттай үед асуудлыг шийдэх гаргалгаа, аргачлалуудыг боловсруулж асуудлыг шийдэж чадсан.

Оюутан Номин-Эрдэнэ нь энэхүү судалгааны ажлыг гүйцэтгэх явцдаа системийг амжилттай бүтээхэд шаардлагатай бүхий л онолын, технологийн мэдлэгтэй болохоо харуулсан. Тэрээр судалгааны арга зүйг сайтар эзэмшсэн, аливаад судалгаа, онолын үндэслэлтэй ханддаг, бичиг баримт боловсруулах чадвар сайтай, бие даан асуудал шийдвэрлэх, шинжилгээ дүгнэлт хийх чадвар бүхий үнэ цэнэтэй төгсөгч гэж үнэлж байна. Түүний хийсэн ажил нь туршилт судалгаа их хийж, цаг хугацаа их зарцуулсан ч хувийн зохион байгуулалт, уйгагагүй хөдөлмөрлж зорьсон зорилгодоо хүрч оновчтой шийдлийг боловсруулсан учраас уг ажилд удирдагчийн зүгээс 100% оноо өгөх саналтай байна.

Ахлах багш **Ж.Үйтүмэн**

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Пүрэвдоржийн Номин-Эрдэнэ

Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)
Бакалаврын судалгааны ажил

Улаанбаатар
2026 оны 3 сар

МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРЫН УХААНЫ ТЭНХИМ

Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл

(Content Analysis and Topic Modeling for Social Monitoring Systems)

Прорграмм хангамж (D061302)
Бакалаврын судалгааны ажил

Удирдагч: Ж.Үйтүмэн

Гүйцэтгэсэн: П.Номин-Эрдэнэ (22b1num4640)

Улаанбаатар
2026 оны 3 сар

Зохиогчийн баталгаа

Миний бие Пүрэвдоржийн Номин-Эрдэнэ ”Сошиал мониторингийн систем дэх агуулгын шинжилгээ, сэдэв загварчлалын шийдэл” сэдэвтэй судалгааны ажлыг гүйцэтгэсэн болохыг зарлаж дараах зүйлсийг баталж байна:

- Ажил нь бүхэлдээ эсвэл ихэнхдээ Монгол Улсын Их Сургуулийн зэрэг горилохоор дэвшүүлсэн болно.
- Энэ ажлын аль нэг хэсгийг эсвэл бүхлээр нь ямар нэг их, дээд сургуулийн зэрэг горилохоор оруулж байгаагүй.
- Бусдын хийсэн ажлаас хуулбарлаагүй, ашигласан бол ишлэл, зүүлт хийсэн.
- Ажлыг би өөрөө (хамтарч) хийсэн ба миний хийсэн ажил, үзүүлсэн дэмжлэгийг дипломын ажилд тодорхой тусгасан.
- Ажилд тусалсан бүх эх сурвалжид талархаж байна.

Гарын үсэг: _____
Огноо: _____

Гарчиг

НЭР ТОМЬЁОНЫ ТАЙЛБАР **1**

УДИРТГАЛ **5**

1 Онолын судалгаа **8**

1.1 Сошиал мониторингийн системийн ерөнхий ойлголт 8

1.2 Судлагдсан байдал 9

1.3 Байгалийн хэлний боловсруулалтын онолын суурь 11

1.4 Сошиал мониторингийн системийн архитектур 15

2 Системийн шинжилгээ, зохиомж **17**

2.1 Системийн шаардлага 17

2.2 Системийн архитектур, зохиомж 21

2.3 Өгөгдлийн сангийн зохиомж 26

2.4 Байгалийн хэлний боловсруулалтын загваруудын сонголт 29

3 Хэрэгжүүлэлт, туршилт **33**

3.1 Ашигласан технологиуд 33

3.2 Өгөгдөл цуглуулалт ба Crawler модулийн хэрэгжүүлэлт 34

3.3 Өгөгдлийн урьдчилсан боловсруулалт 36

3.4 Нэрлэсэн объект таних загварын хэрэгжүүлэлт ба нарийвчилсан сургалт . 38

3.5 Сэдэв загварчлалын хэрэгжүүлэлт 49

3.6 Системийн архитектур: FastAPI болон REST API хэрэгжүүлэлт 55

3.7 Хяналтын самбар болон Хэрэглэгчийн интерфейс 56

3.8 Системийн гүйцэтгэлийн үнэлгээ 58

ДҮГНЭЛТ **60**

НОМ ЗҮЙ **62**

ХАВСРАЛТ **63**

Зургийн жагсаалт

1.1 Сошиал мониторингийн системийн ерөнхий архитектур 15

2.1 Ажлын явцын диаграмм 21
2.2 Системийн ерөнхий архитектур 22
2.3 Системийн давхаргат архитектур 23
2.4 Байгалийн хэлний боловсруулалтын шийдлийн мэдээллийн урсгал 25
2.5 Системийн бүрэлдэхүүн хэсгүүдийн харилцан үйлчлэлийн дараалал 26
2.6 Өгөгдлийн сангийн ER диаграмм 27

Хүснэгтийн жагсаалт

1.1 Сошиал мониторингийн системүүдийн харьцуулсан үзүүлэлт 9

2.1 Системийн функциональ шаардлага 19
2.2 Системийн функциональ бус шаардлага 20
2.3 REST API-ийн үндсэн эндпойнтуудын тодорхойлолт 24
2.4 Өгөгдлийн сангийн хүснэгтүүдийн бүтэц ба зориулалт 28
2.5 Текст боловсруулах хандлагуудын харьцуулалт 30
2.6 NER загваруудын харьцуулалт 31

3.1 Backend технологиуд 33
3.2 Frontend технологиуд 34
3.3 Мэдээний crawler-ийн дэмждэг сайтууд ба аргууд 34
3.4 Preprocessing аргуудын харьцуулалт 38
3.5 HOT суурь загварын гүйцэтгэлийн үнэлгээ (eval/evaluate_tokens.py, 500 нийтлэл) 40
3.6 V1 нарийвчилсан сургалт сургалтын үр дүн (CoNLL word-level тэмдэглэл) 41
3.7 V2 сургалтын өгөгдлийн эх сурвалжууд 42
3.8 V2 сургалтын өгөгдлийн статистик (train_v2_merged.jsonl) 43
3.9 V1 ба V2 сургалтын тохиргооны харьцуулалт 44
3.10 V2 нарийвчилсан сургалт сургалтын үр дүн (тэмдэгт-байрлалын тэмдэглэл) 45
3.11 Суурь загвар, V1, V2 гүйцэтгэлийн харьцуулалт 46
3.12 Суурь загвар ба V2 загварын бодит туршилтын харьцуулалт (5 жишээ) . . 47
3.13 HOT нарийвчилсан сургалт-ийн давуу ба сул тал 49
3.14 Сэдэв загварчлалын аргуудын харьцуулалт 50
3.15 Датасетийн хэмжээнээс хамаарсан сэдвийн тооны тооцоо 53
3.16 BERTopic + KMeans-ийн илрүүлсэн гол сэдвүүд (500 нийтлэл) 54
3.17 BERTopic тохиргооны харьцуулалт — Coherence ба Diversity хэмжүүрүүд (200 нийтлэл) 55
3.18 Модуль бүрийн гүйцэтгэлийн хэмжилт (100 текст, batch) 59

Нэр томьёоны тайлбар

Англи нэр	Монгол орчуулга
Topic Modeling	Сэдэв загварчлал
Web scraping	Веб сайтаас автоматаар мэдээлэл цуглуулах
Sentiment Analysis	Сэтгэл хөдлөлийн шинжилгээ
Named Entity Recognition	Нэрлэсэн объект таних
Embedding	Вектор дүрслэл
POS tagging	Үгсийн аймагт тулгуурлан шошголох
Subword Tokenization	Дэд үгээр токенжуулалт
Pretraining	Урьдчилсан сургалт
Finetuning	Нарийвчилсан сургалт
NLP Pipeline	Байгалийн хэлний боловсруулалтын ажлын урсгал
Preprocessing	Урьдчилсан боловсруулалт
Data Collection	Өгөгдөл цуглуулалт
Text Processing	Өгөгдөл боловсруулалт
Content Analysis	Агуулгын шинжилгээ
Visualization and Reporting	Тайлан ба дүрслэл
Noise	Дуу чимээ
Stream	Мэдээллийн урсгал
Supervised Learning	Удирдлагатай сургалт
Low-latency Inference	Тооцооллын өндөр хурд
Agglutinative	Залгаварт бүтэцтэй
Hyperparameter	Гиперпараметр
Semantic	Үгсийн хоорондын утга зүйн уялдаа холбоо
Supervised Topic Classification	Урьдчилан тодорхойлсон сэдвийн ангилал
Sequence Labeling	Дараалаар шошголох
Classification	Ангилах
Binary Classifier	Хоёртын ангилагч
Multi-label Classifier	Олон шошгот ангилагч
Taxonomy	Таксономи
Self-Attention	Өөртөө анхаарал хандуулах

Англи нэр	**Монгол орчуулга**
Long-range Dependency	Алслагдсан элементүүдийн хоорондын хамаарал
Masked Language Modeling (MLM)	Өгүүлбэр дэх санамсаргүйгээр нуугдсан токенуудыг таах
Next Sentence Prediction (NSP)	Хоёр өгүүлбэрийн логик залгамж холбоог тодорхойлох
One-hot Encoding	Нэг утгат кодчилол
Bag-of-words	Үгсийн уут
BERTopic	BERT embedding болон HDBSCAN clustering ашигладаг орчин үеийн сэдэв загварчлалын арга
Hexagonal Architecture	Портс ба Адаптер архитектур; системийн цөм логикийг гадны хэрэгслээс тусгаарлах зохион байгуулалт
Lazy-load	Шаардлагатай үед ачаалах
Batch inference	Багцлан боловсруулах
Outlier	Гажиг (хэвийн бус) утга
Tokenizer	Токенжуулагч
Dashboard	Хяналтын самбар
Character-offset	Тэмдэгт байрлалд суурилсан
Fallback	Нөөц арга
Stop word	Хасагдах үг (эсвэл сул үг)
Sub-token	Дэд-токен
Unified interface	Нэгдсэн функц (интерфейс)
Two-stage fallback	Хоёр шатлалт нөөц арга

Товчилсон үгс

Товчлол	Нэр томьёо	Монгол тайлбар
LLM	Large Language Model	Том хэлний загвар
CBOM	Continious bag of words	Үргэлжилсэн үгсийн уут
NLP	Natural Language Processing	Эх хэлний боловсруулалт
NER	Named Entity Recognition	Нэрлэсэн объект таних
API	Application Programming Interface	Программчлалын интерфейс
DB	Database	Өгөгдлийн сан
BERT	Bidirectional Encoder Representations from Transformers	Трансформер архитектурт суурилсан хэлний загвар
UMAP	Uniform Manifold Approximation and Projection	Хэмжээс бууруулах арга
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise	Нягтралд суурилсан шатлалт кластерчлал
XLMR	Cross-lingual Language Model RoBERTa	Олон хэлний загвар
GPU	Graphics Processing Unit	Видео карт
CORS	Cross-Origin Resource Sharing	Хөндлөн эх сурвалжийн хандалт хуваалцах
LDA	Latent Dirichlet Allocation	Дирихлегийн далд хуваарилалтын загвар

Техникийн нэр томьёо

Нэр томьёо	Тайлбар
Siamese Network	Ижил жин хуваалцдаг хос мэдрэлийн сүлжээний архитектур; хоёр оролтын ижил төстэй байдлыг хэмжихэд ашиглагддаг.

Нэр томьёо	**Тайлбар**
Apache Kafka	Тархмал урсгалын платформ; өндөр дамжуулалттай мессежний брокер.
Elasticsearch	Тархмал хайлтын болон аналитикийн хөдөлгүүр; бодит цаг хугацаанд их хэмжээний өгөгдлийг индексжүүлж, хурдан хайлт болон шинжилгээ хийх боломжийг олгодог.

Удиртгал

Орчин үед байгууллага, компаниуд хэрэглэгчдийн санал сэтгэгдлийг сонсох, тренд болж буй сэдвүүдийг илрүүлэх зорилгоор сошиал мониторингийн системийг өргөнөөр ашиглах болсон. Одоогийн байдлаар олон улсын сошиал мониторингийн системүүд нь ихэвчлэн түлхүүр үгт тулгуурласан эсвэл энгийн ангиллын аргуудыг ашигладаг бөгөөд агуулгын гүнзгий задлан шинжилгээ, бүтэцчилсэн сэдвийн илрүүлэг хангалттай биш байна. Мөн их хэмжээний өгөгдөл дээр том хэлний загвар шууд ашиглах нь тооцооллын зардал их шаардах ба удаан (latency өндөр) байдаг. Иймд Монгол хэл, тэр дундаа сошиал өгөгдөл дээр үр дүнтэй ажиллаж чадах эх хэлний боловсруулалтад суурилсан шийдэл боловсруулах шаардлага тулгарч байна.

Тухайлбал Нэрлэсэн объект таних болон Сэдэв загварчлалын нэгдсэн судалгааны ажлын урсгал боловсруулах шаардлага тулгарч байна.

Судалгааны үндэс, асуудлын тодорхойлолт

Сошиал мониторингийн системүүд өнөөдөр дараах үндсэн сорилттай тулгарч байна. Үүнд:

1. **Гүнзгий агуулгын шинжилгээ дутмаг:** Түлхүүр үгт суурилсан аргууд нь тухайн нийтлэл *юуны тухай* яриад байгааг буюу сэдвийг автоматаар ялгаж чаддаггүй. Жишээлбэл, *”Монгол улс Оростой хэлэлцээ хийв”* гэсэн мэдээний голлох сэдэв, оролцогч талуудыг автоматаар ялгаж гаргах боломж хязгаарлагдмал.

2. **Том хэлний загварын тооцооллын өртөг:** Их хэмжээний өгөгдөл дээр GPT-4 зэрэг том хэлний загварын программчлалын интерфейс ашиглах нь 100,000 текст дээр ойролцоогоор $200–$300 зарцуулдаг бөгөөд хариу авах хугацаа 1–3 секунд болдог.

3. **Монгол хэлний эх хэлний боловсруулалтын дэмжлэг дутмаг:** Монгол хэл нь залгаварт шинж чанартай бөгөөд нэг үгийн олон хэлбэр, галиг бичлэгийн олон хувилбар, ярианы хэллэгийн онцлог зэргээс шалтгаалан уламжлалт эх хэлний боловсруулалтын хэрэгслүүд бага нарийвчлалтай ажилладаг.

Эдгээр асуудлыг шийдвэрлэхийн тулд шийдлийн гурван аргыг харьцуулан судалсан. Тодруулбал:

- **Мод бүтэцтэй мэдлэгийн сан үүсгэх:** Хамгийн их давтагдаж буй токенуудыг гараар ангилах замаар шаталсан мод бүтэц бүхий мэдлэгийн санг өгөгдлийн санд үүсгээд шинэ мэдээлэл орж ирэх үед тухайн мэдлэгийн сангаас хайж, байхгүй бол шинээр нэмнэ. Энэ арга нь GPU шаардахгүй, алдаа багатай боловч гараар ангилах их хугацаа орж, шинэ үг гарвал гараар нэмэх шаардлагатай.
- **Урьдчилан сургасан BERT загварыг нарийвчилан сургах:** BERT зэрэг урьдчилан сургасан загварыг монгол хэлний сошиал өгөгдөл дээр нарийвчлан сургаж ашиглах арга юм. GPU дэмжлэгтэй T4 дээр 200–500 өгүүлбэр/секунд гүйцэтгэлтэй.
- **Том хэлний загварын программчлалын интерфейс ашиглах:** GPT-4o зэрэг том хэлний загварын программчлалын интерфейс ашиглах арга юм. Энэ нь сургалт шаардалгүй, хэрэглэхэд хялбар уян хатан боловч зардал өндөр, интернет холболтгүйгээр ашиглах боломжгүй.

Судалгааны зорилго, зорилт

Зорилго: Монгол хэлний сошиал мониторингийн системд зориулсан Нэрлэсэн объект таних болон Сэдэв загварчлалд суурилсан эх хэлний ажлын урсгалын шийдэл боловсруулж, Монгол хэлний онцлогт тохирсон аргачлалыг судлан турших, гүйцэтгэлийн үнэлгээ хийхэд оршино.
Зорилт: Доорх зорилтыг тавьж ажиллав. Үүнд:

1. Ижил чиглэлийн олон улсын системүүдийн эх хэлний боловсруулалтын архитектур, аргачлалыг судалж, харьцуулан шинжлэх.
2. Монгол хэлний сошиал өгөгдөлд тохирсон урьдчилсан боловсруулалтын аргачлал боловсруулах.
3. `Davlan/bert-base-multilingual-cased-ner-hrl` суурь загварыг Монгол хэлний өгөгдөл дээр нарийвчлан сургаж, нэрлэсэн объект таних нарийвчлалыг дээшлүүлэх.
4. BERTopic + KMeans болон LDA аргуудыг Монгол текстэд хэрэглэж, харьцуулалт хийх.
5. Судалгааны үр дүнг дүрслэн харуулах туршилтын вэб интерфейс хөгжүүлэх.
6. Монгол хэлний эх хэлний боловсруулалтанд зориулсан нээлттэй, дахин ашиглах боломжтой модульчлагдсан архитектур хэрэгжүүлэх.

Судалгааны арга зүй, хамрах хүрээ, хязгаарлалт

Судалгааны арга зүй: Судалгаанд чанарын болон тоон шинжилгээний аргуудыг хослуулан ашиглав. Үндсэн арга зүй нь баримт бичгийн судалгаа, алгоритмийн харьцуулалт, болон бодит Монгол мэдээний өгөгдөл дээрх туршилт дээр суурилсан.

Баримт бичгийн судалгааны хүрээнд эх хэлний боловсруулалтын суурь онол, BERTopic, BERT, XLMR загварууд болон ижил системүүдийн (Brandwatch, Meltwater, Sprout Social) архитектур, аргачлалын судалгаа хийсэн.

Туршилтын хүрээнд Монгол мэдээний нийтлэл дээр Нэрлэсэн объект таних сэдэв загварчлалын аргачлалуудыг туршиж, загваруудын гүйцэтгэлийг тоон хэмжүүрүүдээр үнэлэн харьцуулсан.

Судалгааны хүрээ: Энэхүү судалгааны ажил нь монгол хэл дээрх мэдээ мэдээлэл болон нийгмийн сүлжээний өгөгдөлд эх хэл боловсруулалтын аргуудыг турших, хэрэгжүүлэх асуудалд төвлөрсөн. Судалгааны үндсэн чиглэл нь монгол хэлний өвөрмөц онцлогт тохирсон нэрлэсэн объект таних загварыг нарийвчлан сургах болон ”BERTopic” алгоритмд суурилсан сэдэв загварчлалын шийдэл боловсруулахад чиглэгдсэн. Системийн хөгжүүлэлт, туршилтад Python хэл, Hugging Face Transformers, BERTopic, sentence-transformers сангуудыг ашигласан бөгөөд архитектурын шийдлийг FastAPI, Next.js болон SQLite технологиуд дээр суурилан гүйцэтгэв.

Судалгааны хязгаарлалт: Судалгааны ажлын үр дүн болон хэрэгжилт нь дараах хүчин зүйлсээр хязгаарлагдана. Үүнд:

- **Өгөгдлийн төрөл зүйл:**Туршилт, шинжилгээ нь зөвхөн монгол хэл дээрх цахим мэдээ болон сошиал медиагийн өгөгдөлд суурилсан тул үр дүнг бусад хэлний эх бичвэр эсвэл өөр салбарын (хууль, анагаах ухаан г.м) төрөлжсөн текстэд шууд хамаатуулах боломжгүй.

- **Түүвэр өгөгдлийн хэмжээ:** NER загварын үнэлгээнд ашигласан гар тэмдэглэгээт датасет нь 915 өгүүлбэрийн хүрээнд хязгаарлагдаж байгаа нь загварын ерөнхийлөх чадварт нөлөөлж болзошгүй. Цаашид сургалтын өгөгдлийн санг баяжуулснаар загварын нарийвчлал болон найдвартай байдлыг статистикийн ач холбогдол бүхий түвшинд дээшлүүлэх шаардлагатай.

Бүлэг 1

Онолын судалгаа

1.1 Сошиал мониторингийн системийн ерөнхий ойлголт

Сошиал мониторингийн систем нь олон нийтийн сүлжээ, мэдээллийн портал, блог болон бусад цахим платформуудаас өгөгдөл цуглуулах, боловсруулах, дүн шинжилгээ хийх замаар бодит цагийн мэдээллийг гарган авах автоматжуулсан цогц үйл явц юм [7]. Орчин үеийн сошиал мониторингийн системүүд нь ерөнхийдөө дараах үндсэн үе шатуудыг дамжин хэрэгждэг. Үүнд:

1. **Өгөгдөл цуглуулалт:** Программчлалын интерфейс, веб сайтаас автоматаар мэдээлэл цуглуулах болон RSS зэрэг технологи ашиглан олон эх сурвалжаас бодит цагт өгөгдөл татах.

2. **Текст боловсруулалт:** Цугласан өгөгдлийг цэвэрлэж, стандартчилж, шинжилгээнд бэлтгэх.

3. **Агуулгын шинжилгээ:** Эх хэлний боловсруулалтын технологи ашиглан текстийн сэдэв, сэтгэл хөдлөл, оролцогч талуудыг тодорхойлох.

4. **Тайлан ба Дүрслэл:** Үр дүнг хэрэглэгчид ойлгомжтой хэлбэрт оруулан харуулах.

Их өгөгдлийн боловсруулалтын сорилтууд

Интернэтийн хэрэглэгчдийн тоо 2024 оны байдлаар 5.4 тэрбумд хүрч, өдөрт 500 сая жиргээ, 350 сая пост зэрэг хэмжээлшгүй их өгөгдөл үүсч байна [13]. Энэхүү их өгөгдлийн боловсруулалтад дараах үндсэн сорилтууд тулгарч байна. Үүнд:

- **Хурд:** Өгөгдөл асар өндөр хурдаар үүсэж байгаа тул түүнийг бодит цагийн горимд боловсруулах, хариу үйлдэл үзүүлэх шаардлагатай болдог.

- **Өгөгдлийн хэмжээ:** Тэрбумаар хэмжигдэх текстэн өгөгдлийг үр ашигтай хадгалах, задлан шинжлэх зэрэг дэд бүтцийн асуудал тулгардаг.

- **Өгөгдлийн олон талт байдал:** Бүтэцлэгдсэн(өгөгдлийн сангийн хүснэгт), хагас бүтэцлэгдсэн(JSON, XML) болон бүтэцлэгдээгүй (текст) өгөгдлийн хэлбэрүүдийг нэгтгэн боловсруулах шаардлага үүсдэг.
- **Үнэн зөв байдал:** Нийгмийн сүлжээний өгөгдөл нь дуу чимээ ихтэй буюу спам, хуурамч мэдээлэл, давтагдсан контент агуулдаг тул өгөгдлийн үнэн зөв, найдвартай байдлыг хангах нь чухал юм.

Дээрх сорилтуудыг амжилттай шийдвэрлэхэд Байгалийн хэл боловсруулалтын технологиуд, тэр дундаа гүн сургалтын аргуудад суурилсан алгоритмуудын гүйцэтгэх үүрэг, ач холбогдол улам бүр нэмэгдсээр байна.

1.2 Судлагдсан байдал

Ижил системүүдийн судалгаа

Сошиал мониторингийн чиглэлээр тэргүүлж буй системүүдийн архитектур, өгөгдөл боловсруулах арга зүй нь энэхүү судалгааны чухал суурь болж байна.

Brandwatch платформ нь Apache Kafka-д суурилсан бодит цагийн мэдээллийн урсгал боловсруулах аргачлалыг ашигладаг. Текст ангилалд LSA болон хяналттай машин сургалтын аргуудыг хослуулдаг ч [3] Монгол хэлний морфологийн онцлогт тохирсон хэл шинжлэлийн боловсруулалт дутмаг, нэвтрүүлэлтийн зардал өндөр зэрэг сул талтай.

Meltwater нь Elasticsearch-т суурилсан индексжүүлэлтээр 270 гаруй хэлний мэдээллийг бодит цагт шүүдэг. Сэтгэл хөдлөлийн шинжилгээнд машин сургалтын хоёртын ангилагч ашигладаг бол сэдэв илрүүлэлтэд LDA загварт суурилсан ангиллын таксономиг хэрэглэдэг [8].

Sprout Social систем нь байгалийн хэлний боловсруулалтандаа spaCy болон Transformers архитектурыг нэгтгэсэн бөгөөд олон шошгот ангиллын аргаар нийтлэлийг ангилдаг.

Эдгээр системүүдийн харьцуулалтаас харахад (Хүснэгт 1.1), Монгол хэлний залгаварт бүтцэд тохирсон, тооцооллын өндөр хурд бүхий шийдлүүд дутагдалтай хэвээр байна.

Систем	**NLP алгоритм**	**Монгол хэл**	**Нээлттэй эх**
Brandwatch	LSA + ML	Дэмждэггүй	Үгүй
Meltwater	LDA + Rules	Хязгаарлагдмал	Үгүй
Sprout Social	Transformers	Дэмждэггүй	Үгүй
Sumbee.mn	Keyword-based	Бүрэн	Үгүй
Энэхүү судалгаа	**BERT + BERTopic**	**Бүрэн**	**Тийм**

Хүснэгт 1.1: Сошиал мониторингийн системүүдийн харьцуулсан үзүүлэлт

Монгол хэлний Байгалийн хэлний боловсруулалтын судлагдсан байдал

Дэлхий нийтэд байгалийн хэлний боловсруулалтын технологи эрчимтэй хөгжиж, том хэлний загварууд шинэ шатанд гарч байгаа хэдий ч Монгол хэлний хувьд хэл шинжлэлийн өвөрмөц тогтолцоо болон дижитал нөөц хомс байгаагаас шалтгаалан тусгайлсан судалгаа, арга зүйг шаардсаар байна. Монгол хэлний Байгалийн хэлний боловсруулалтын өнөөгийн түвшин болон тулгарч буй сорилтуудыг дараах үндсэн чиглэлүүдээр тодорхойлж болно.Үүнд:

Морфологийн онцлог: Монгол хэл нь *залгаварт* бүтэцтэй бөгөөд нэг үндсэн үгнээс олон зуун хэлбэр үүсэх боломжтой. Жишээлбэл, “цахилгааанжуулалтынхантайгаа” гэсэн нэг үг нь үгийн үндэс болон залгаврууд ын нийлбэрээс бүрддэг. Энэ онцлог нь уламжлалт токенжуулах аргу удыг үр дүнгүй болгодог [1].

Галиг бичлэгийн олон хувилбар: Сошиал орчинд Монгол хэрэглэгчид латин үсгийн галигийг өргөнөөр ашигладаг. “Монгол”, “Mongol”, “mongl”, “мgl” зэрэг олон хэлбэрийг нэг утгаар таних шаардлагатай байдаг.

Академик судалгааны байдал: Mongolian NLP Corpus [10] нь 1.2 сая өгүүлбэртэй текстийн сан бүрдүүлсэн бол Ж.Пүрэв болон Ч.Одбаяр нар [11] Монгол хэлний морфологийн шинжилгээнд суурилсан үгсийн аймагт тулгуурлан шошголох системийг боловсруулсан. Сошиал медиагийн чиглэлд Д.Энхзол [**?**] Монгол жиргээний (tweet) сэтгэгдлийн мэдрэмжийн шинжилгээнд мэдрэлийн сүлжээ болон Text2Plot аргыг ашигласан судалгаа хийсэн бөгөөд зорилтот хандлагын шинжилгээ чиглэлд хувь нэмэр оруулсан. Харин сошиал өгөгдөлд зориулсан HOT болон сэдэв загварчлалын судалгаа маш хязгаарлагдмал хэвээр байна.

Олон хэлт загваруудын хувь нэмэр: XLM-RoBERTa [4], mBERT [5] зэрэг олон хэлт урьдчилан сургасан загварууд нь монгол хэл дээр нарийвчлан сурган ашиглах боломжтой болсон нь том ахиц юм.

Сэдэв загварчлалын судалгааны хөгжил

Сэдэв загварчлалын чиглэлд Д.Блей болон бусад судлаачдын 2003 онд санал болгосон Дирихлегийн далд хуваарилалтын загвар [2] нь магадлалын статистик загварчлалын суурь арга болж тогтсон. Гэвч Дирихлегийн далд хуваарилалтын загвар нь үгсийн утга зүйн хамаарал болон контекстийг бүрэн илэрхийлж чаддаггүй сул талтай. Үүнийг шийдвэрлэх дараагийн томоохон ахиц нь 2022 онд М.Грутендорстын санал болгосон BERTopic [6] загвар юм. Энэхүү арга нь Transformer архитектурт суурилсан контекст векторуудыг хэмжээс бууруулах UMAP алгоритм болон нягтралд суурилсан кластерчлалын HDBSCAN алгоритмын аргуудтай хослуулснаар LDA-аас илүү өндөр нарийвчлалтай, утга зүйн хувьд уялдаатай сэдвүүдийг илрүүлэх боломжийг бүрдүүлсэн.

1.3 Байгалийн хэлний боловсруулалтын онолын суурь

Transformer архитектурт суурилсан BERT загвар

2017 онд Google Brain-ийн эрдэмтэд “Attention Is All You Need” [14] өгүүлэлдээ *Transformer* архитектурыг танилцуулсан нь Байгалийн хэлний боловсруулалтын хөгжилд маш том хувьсгал болсон. Transformer нь RNN-ийн дараалалт боловсруулалтын оронд *self-attention* механизмыг ашиглан өгүүлбэрийн бүх токенуудын хоорондын хамаарлыг нэгэн зэрэг тооцдог болсон нь дараах давуу талуудыг бий болгосон. Үүнд:

- **Параллель тооцоолол:** Өгүүлбэрийн токенуудыг нэгэн зэрэг боловсруулснаар сургалтын хугацааг RNN-ээс хэд дахин богиносгосон;
- **Алслагдсан элементүүдийн хоорондын хамаарал:** Өгүүлбэрийн эхлэл болон төгсгөл дэх үгсийн хамаарлыг алдалгүйгээр тооцоолох чадамж;
- **Динамик дүрслэл:** Үг бүрийн утгыг тухайн өгүүлбэр дэх бусад бүх үгтэй нь харьцуулан контекст хамаарал бүхий вектор болгон дүрсэлдэг.

2019 онд танилцуулагдсан BERT [5] загвар нь Transformer-ийн энэхүү чадавхыг ашиглан хоёр талт контекстийг сурдаг. BERT нь өгүүлбэр дэх нуугдсан үгийг таах, дараагийн өгүүлбэрийг таах даалгавруудыг гүйцэтгэснээр хэлний гүнзгий мэдлэгийг өөртөө шингээдэг. Загварын үндсэн механизмыг дараах байдлаар томьёолно:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{1.1}$$

Үүнд Q (Query), K (Key), V (Value) нь оролтын өгөгдлийн шугаман хувиргалтууд бөгөөд d_k нь Key-ийн хэмжээс буюу масштаблах хүчин зүйл болно.

Текстийн вектор дүрслэл

Текстийн вектор дүрслэл нь үг, өгүүлбэр болон баримт бичгийг тасралтгүй тоон векторт хөрвүүлэх арга юм. Уламжлалт үгсийн уут аргаас ялгаатай нь семантик утгыг хадгалдаг ба утга ойролцоо үгс тоон орон зайд ойр байрладаг. Бичвэрийг тоон хэлбэрт шилжүүлэх анхны аргууд болох нэг-утгат кодчилол эсвэл үгсийн уут нь үгсийн хоорондын утга зүйн уялдааг илэрхийлж чаддаггүй байв. Энэхүү хязгаарлагдмал байдлыг шийдвэрлэхээр текстийн семантик шинж чанарыг олон хэмжээст огторгуйд дүрслэх боломжтой вектор дүрслэлийн аргууд хөгжиж эхэлсэн юм. Тухайлбал:

Word2Vec [9]: 2013 онд Т.Миколов болон Google-ийн судлаачдын санал болгосон энэхүү арга нь байгалийн хэл боловсруулалтын салбарт векторт суурилсан сургалтын шинэ эрин үеийг нээсэн. Уг загвар нь хоёр давхаргат нейрон сүлжээний тусламжтайгаар үгсийн контекст хамаарлыг Skip-gram болон үргэлжилсэн үгсийн уут архитектураар дамжуулан сурдаг. Word2Vec нь үгсийн сангийн утга зүйн хамаарлыг векторын

арифметик үйлдлээр илэрхийлэх боломжийг бүрдүүлсэн анхны томоохон шийдэл юм. **Sentence-BERT (SBERT)** [12]: Уламжлалт BERT загвар нь өгүүлбэр хоорондын ижил төстэй байдлыг тооцоолохдоо нүсэр тооцоолол шаарддаг сул талтай. Үүнийг шийдвэрлэхээр SBERT загвар нь BERT-ийн бүтцэд *siamese network* архитектурыг нэгтгэж, өгүүлбэрийн утга зүйн мэдээллийг хадгалсан оновчтой векторуудыг үүсгэдэг. Энэхүү арга нь өгүүлбэр хоорондын семантик ижил төстэй байдлыг косинус төсөөтэй байдлын томьёогоор бодит цагийн горимд, үр ашигтай хэмжих боломжийг олгодог

$$\text{sim}(u, v) = \frac{u \cdot v}{|u| \cdot |v|} \tag{1.2}$$

Үүнд $u, v \in \mathbb{R}^d$ нь харгалзах текстийн векторууд болно.Энэхүү судалгааны хүрээнд Монгол хэл дээрх сошиал болон мэдээллийн текстийг векторжуулахад олон хэлний контекст хамаарлыг өндөр нарийвчлалтай илэрхийлдэг `paraphrase-multilingual-mpnet-base-v2` загварыг сонгон ашиглав. Уг загвараар үүсгэсэн семантик векторууд нь сэдвийн ангилал болон бичвэрүүдийн ижил төстэй байдлын шинжилгээнд тооцооллын үндсэн оролт болж ашиглагдсан. Текстийг ийнхүү вектор орон зайд дүрсэлсний дараах дараагийн чухал алхам бол өгөгдлийн дотоод бүтэц, утга зүйн зангилааг илрүүлэх үйл явц юм. Үүний тулд их хэмжээний текстээс далд орших сэдвүүдийг автоматаар ялгах сэдэвчилсэн загварчлалын аргуудыг ашигладаг.

Сэдэв загварчлал

Сэдэв загварчлал нь бүтэцлэгдээгүй бичвэрүүдээс утга зүйн ижил төстэй байдлаар бүлэглэгдсэн ”далд” сэдвүүдийг илрүүлэх хяналтгүй сургалтын арга юм. Энэхүү судалгаанд уламжлалт статистик загварчлал болон орчин үеийн векторт суурилсан аргуудыг харьцуулан авч үзнэ. Үүнд:

Дирихлегийн далд хуваарилалтын загвар

LDA [2] нь Байесын магадлалын онолд суурилсан генератив загвар бөгөөд баримт бичиг бүрийг сэдвүүдийн холимог, сэдэв бүрийг үгсийн магадлалт тархалт гэж үздэг. Загварын үндсэн логик нь өгөгдсөн бичвэрүүдийн цаана орших далд бүтцийг дараах магадлалын томьёоллоор тодорхойлоход оршино.

$$p(\mathbf{w}, \mathbf{z}, \theta, \phi|\alpha, \beta) = \prod_{k=1}^{K} p(\phi_k|\beta) \prod_{d=1}^{D} p(\theta_d|\alpha) \prod_{n=1}^{N_d} p(z_{dn}|\theta_d) p(w_{dn}|\phi_{z_{dn}}) \tag{1.3}$$

Үүнд θ_d нь d-р баримт бичгийн сэдвийн тархалт, ϕ_k нь k-р сэдвийн үгсийн тархалт, харин α болон β нь Дирихлегийн тархалтын гиперпараметрүүд юм. LDA загварын гол давуу тал нь үр дүнг тайлбарлах боломж өндөртэй бөгөөд тооцооллын нөөц бага шаарддагт оршино. Гэсэн хэдий ч уг арга нь сэдвийн тоог урьдчилан зааж өгөх шаардлагатайгаас гадна үгсийн утга зүйн хамаарлыг тооцдоггүй үгсийн уут зарчим дээр суурилдаг. Иймд

сошиал медиагийн богино бичвэрүүд болон спам ихтэй өгөгдөлд ажиллах чадвар нь харьцангуй сул байдаг хязгаарлалттай.

BERTopic

LDA загварын хязгаарлагдмал байдлыг шийдвэрлэх орчин үеийн шийдэл бол Transformer архитектурт суурилсан BERTopic [6] загвар юм. Энэхүү арга нь текстийн семантик утгыг хэмжээст огторгуйд хадгалж, кластерчлалын алгоритмаар дамжуулан сэдвүүдийг илрүүлдэг. BERTopic загварчлал нь дараах үндсэн дөрвөн үе шатаас бүрдэнэ.Үүнд:

1. **Вектор дүрслэл:** *Sentence-BERT* загварыг ашиглан бичвэр бүрийг утга зүйн мэдээлэл агуулсан өндөр хэмжээст (ихэвчлэн $\mathbb{R}^{384}$ эсвэл $\mathbb{R}^{768}$) вектор болгон хөрвүүлнэ.

2. **Хэмжээс бууруулалт:** Өндөр хэмжээст векторууд дээр кластерчлал хийхэд үүсэх хэмжээсийн төвөгшлөөс сэргийлэхийн тулд UMAP алгоритмаар хэмжээсийг багасгаж, өгөгдлийн дотоод бүтцийг хадгалсан бага хэмжээст орон зайд шилжүүлнэ.

3. **Кластерчлал:** Нягтралд суурилсан кластерчлалын HDBSCAN алгоритмыг ашиглан ижил утгатай бичвэрүүдийг бүлэглэнэ. Энэхүү алгоритм нь сэдвийн тоог урьдчилан заах шаардлагагүй бөгөөд утга зүйн хувьд ямар нэгэн бүлэгт хамаарахгүй дуу чимээг бие даан ялгах давуу талтай.

4. **Сэдвийн дүрслэл:** Кластер бүрийг төлөөлөх түлхүүр үгсийг ялгахын тулд уламжлалт TF-IDF-ийн сайжруулсан хувилбар болох класст суурилсан c-TF-IDF аргыг ашигладаг.

c-TF-IDF-ийн тооцооллын томьёог доорх байдлаар тодорхойлно:

$$W_{t,c} = tf_{t,c} \cdot \log\left(1 + \frac{A}{tf_t}\right) \tag{1.4}$$

Үүнд $tf_{t,c}$ нь c кластер дахь t үгийн давтамж, A нь баримт бичгийн дундаж тоо, tf_t нь нийт кластеруудын хэмжээнд тухайн үгийн гарсан давтамж юм. Энэхүү хэмжигдэхүүн нь тухайн сэдвийг бусдаас ялгах хамгийн өндөр ач холбогдол бүхий үгсийг тодорхойлдог.

Нэрлэсэн объект таних

НОТ нь текст дэх бүтэцлэгдээгүй өгөгдлөөс урьдчилан тодорхойлсон ангилал бүхий объектыг (хүн, байгууллага, байршил, цаг хугацаа г.м) автоматаар илрүүлж, ангилах даалгавар юм. Орчин үеийн сошиал мониторингийн системд энэ нь хэн, хаана, юуны тухай ярьж байгааг тодорхойлох суурь алхам болдог НОТ-ийг системд нэвтрүүлэхийн тулд хамгийн түрүүнд текст дэх чөлөөт бичвэрийг компьютерт боловсруулагдах боломжтой, дискрет бүтцэд шилжүүлэх шаардлагатай болдог. Өөрөөр хэлбэл, энгийн өгүүлбэрийг машин сургалтын загварт оролт болгон өгөхөд тохиромжтой математик

төлөөлөл болон стандартчилсан шошгоор баяжуулах үйл явц юм. Энэхүү хөрвүүлэлтийг гүйцэтгэхдээ бид өгөгдлийг тодорхой форматын дагуу загварчилж, токен бүрийн байрлал болон үүргийг тэмдэглэсэн тусгай схемийг ашигладаг. Тухайлбал: **Дараалсан шошгололт**НОТ нь мөн чанартаа дараалсан шошгололтын асуудал бөгөөд оролт болон ирж буй токен бүрийг харгалзах ангиллын шошгоор тэмдэглэдэг. Энэхүү даалгаварт BIO (Begin-Inside-Outside) схемийг CoNLL-2003 стандартын дагуу өргөнөөр ашигладаг. **CoNLL формат:**Энэ нь NER-ийн өгөгдлийг баганаар ангилсан формат бөгөөд токен бүр өөрийн харгалзах шошгын хамт нэг мөрөнд бичигддэг. Жишээ нь, ”МУИС Улаанбаатарт байрладаг” гэсэн өгүүлбэрийг CoNLL форматаар үзүүлбэл:

- `МУИС ORG B-ORG`
- `Улаанбаатарт LOC B-LOC`
- `байрладаг O O`

BIO Tagging schema:Текст доторх биетийн хил хязгаарыг нарийн тогтоохын тулд дараах шошгоор токенжуулдаг:

- `B-TYPE`: Тухайн нэрлэсэн биетийн эхний токен (жишээ нь: `B-PER`).
- `I-TYPE`: Тухайн биетийн доторх буюу үргэлжлэл токенууд (жишээ нь: `I-PER`).
- `O (Outside)`: Ямар нэгэн нэрлэсэн биетэд хамаарахгүй ердийн токен.

BERT-д суурилсан нэрлэсэн биет таних

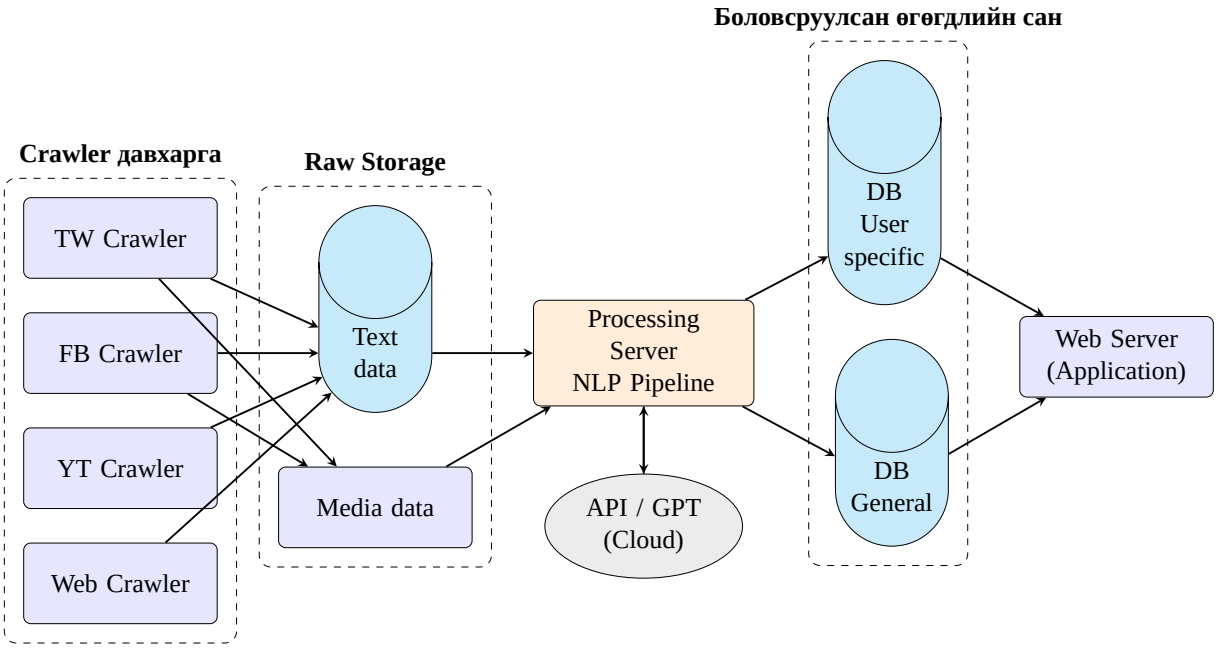
Уламжлалт статистик аргуудаас ялгаатай нь BERT-д суурилсан шийдэл нь текстийн хоёр талын контекстийг бүрэн ашигладаг. Загварын сүүлчийн давхаргад токен бүрийн хувьд $softmax$ ангилагч ажиллуулж, тухайн токен нь аль BIO шошгонд хамаарах магадлалыг дараах байдлаар тооцоолно:

$$p(y_i|x_i, \mathbf{h}) = \text{softmax}(W \cdot h_i + b) \tag{1.5}$$

Энд h_i нь i-р токений BERT-ийн контекст хамааралтай нууц төлөв (*hidden state*), W ба b нь сургагдах жингийн матриц болон хазайлтын параметрүүд юм. Энэхүү судалгаанд `Davlan/bert-base-multilingual-cased-ner-hrl` загварыг суурь болгон ашигласан. Уг загвар нь 10 гаруй хэлний CoNLL-2003 болон WikiAnn зэрэг томоохон хэмжээний стандарт өгөгдлийн багцууд дээр урьдчилан сургагдсан олон хэлт загвар юм. Энэхүү загварыг сонгосон нь Монгол хэл шиг бага нөөцтэй хэлний хувьд тусгайлан нарийвчилан сургах замаар өндөр нарийвчлал үзүүлэх боломжийг бүрдүүлж байна.

1.4 Сошиал мониторингийн системийн архитектур

Орчин үеийн сошиал мониторингийн систем нь асар их хэмжээний бүтэцлэгдээгүй өгөгдлийг бодит цагийн горимд цуглуулах, цэгцлэх, дүн шинжилгээ хийх зориулалттай нарийн төвөгтэй цогц шийдэл юм. Системийн тогтвортой ажиллагаа болон өгөгдөл боловсруулах хурдыг хангахын тулд архитектурыг модульчлагдсан байдлаар зохиомжилдог. Зураг 1.1-д сошиал мониторингийн системийн ерөнхий бүтцийг давхарга бүрээр ангилан үзүүлэв.



Зураг 1.1: Сошиал мониторингийн системийн ерөнхий архитектур

Системийн үндсэн бүрэлдэхүүн хэсгүүдийг дараах байдлаар тодорхойлж болно:

1. **Өгөгдөл цуглуулах давхарга:** Төрөл бүрийн эх сурвалжаас (Twitter/X, Facebook, YouTube, вэб сайтууд) өгөгдлийг нэгэн зэрэг цуглуулах үүрэгтэй.

2. **Боловсруулаагүй өгөгдлийн хадгалах давхарга:** Өгөгдөл цуглуулах хэсгээс ирж буй текст болон медиа (зураг, видео) өгөгдлийг боловсруулахаас өмнө анхны байдлаар нь хадгална. Ингэснээр өгөгдөл алдагдахаас сэргийлэх ба дахин боловсруулалт хийх боломжийг бүрдүүлдэг.

3. **Өгөгдөл боловсруулах давхарга:** Системийн хамгийн чухал хэсэг бөгөөд цугласан өгөгдлийг эх хэл боловсруулалтын шийдлийг ашиглан боловсруулна. Үүнд: текст цэвэрлэгээ, нэрлэсэн объекь таних, сэдэв тодорхойлох болон сэтгэл хөдлөлийн шинжилгээ зэрэг багтана. Мөн нөөц их шаардах тооцоололд гадаад том хэлний загварын программчлалын интерфейсийг хослуулан ашиглах боломжтой.

4. **Боловсруулсан өгөгдөл хадгалах давхарга:** NLP дамжлагаар шүүгдэж, бүтэцлэгдсэн өгөгдөл (JSON/Relational) хадгалагдах хэсэг юм. Энд хэрэглэгчийн

тохиргоо болон системийн ерөнхий статистик өгөгдлүүд тусдаа баазуудад хуваарилагдана.

5. **Хэрэглээний давхарга:** Боловсруулсан өгөгдлийг хэрэглэгчдэд ойлгомжтой график, тайлан, хяналтын самбар хэлбэрээр үзүүлэх вэб интерфейс юм.

Одоогийн системүүдийн эх хэлний боловсруулалтын хязгаарлалт

Дээрх архитектурт дурдсан Өгөгдөл боловсруулах давхарга нь системийн үнэ цэнийг тодорхойлогч гол хүчин зүйл боловч өнөөгийн ихэнх системүүдэд дараах техникийн сорилтууд тулгарсаар байна. Текст боловсруулалтад ихэвчлэн түлхүүр үгэнд суурилсан болон дүрэмд суурилсан аргуудыг ашиглаж байгаа нь дараах хязгаарлалтуудыг бий болгодог. Үүнд:

- **Контекст хамааралгүй сэдэв илрүүлэлт:** Текстэд дурдагдаж буй үгсийн утга зүйн хамаарлыг тодорхойлж чаддаггүй тул нийтлэлийн үндсэн агуулгыг автоматаар ангилах боломж хязгаарлагдмал.

- **Нэрлэсэн объектын харилцааны шинжилгээ:** Текст дэх хүн, байгууллага, байршлуудыг танихаас гадна тэдгээр нь хоорондоо ямар сэдвийн хүрээнд холбогдож байгааг тодорхойлох алгоритм дутмаг байна.

- **Тооцооллын зардал болон хоцролт (Latency):** GPT-4 зэрэг томоохон хэлний загваруудыг их хэмжээний өгөгдөл дээр шууд ашиглах нь санхүүгийн хувьд өндөр зардалтай (жишээ нь: 100,000 текст тутамд $200 - -300$) байна.

Энэхүү судалгааны ажил нь дээрх хязгаарлалтуудыг шийдвэрлэхэд чиглэсэн бөгөөд ялангуяа Монгол хэлний өгөгдөл дээр НОТ болон Сэдэвчилсэн загварчлалыг өндөр нарийвчлалтай, зардал багатай гүйцэтгэх оновчтой шийдлийг боловсруулахад анхаарсан болно.

Онолын дүгнэлт

Энэ бүлэгт Transformer архитектурт суурилсан BERT загвар, BERTopic болон LDA-д суурилсан сэдэв загварчлал, нэрлэсэн объект таних аргачлал зэрэг системийн NLP цөм болох онолын суурийг авч үзэв. Монгол хэлний залгаварт морфологи нь уламжлалт дүрэм болон статистикийн аргуудын практик хэрэглээнд хязгаарлагдмал байдал үүсгэж, тусгайлсан шийдэл шаарддаг болохыг тодорхойлов. Урьдчилан сургасан олон хэлний загваруудын дэд үгийн токенжуулалт нь энэ сорилтыг хэлний аргаар шийдэх боломж нээдэг бол мэдлэгийн санд суурилсан арга нь домайн бүрийн тусгай нэршлийг нарийвчлан таних нөхөн давуу талтай. Онолын энэхүү суурьт тулгуурлан дараагийн бүлэгт системийн зохиомж болон тодорхой технологийн сонголтыг нотолгоотойгоор авч үзнэ.

Бүлэг 2

Системийн шинжилгээ, зохиомж

Энэхүү судалгааны хүрээнд боловсруулсан байгалийн хэл боловсруулалтын ажлын урсгалыг нээлттэй, уян хатан бөгөөд дахин ашиглагдах боломжтой байхаар зохиомжлов. Системийг судлаачид болон хөгжүүлэгчид дараах хоёр үндсэн хэлбэрээр ашиглах боломжтой. Үүнд:

- **Demo Web Application:** Судалгааны туршилтын үр дүнг интерактив байдлаар харуулах, текст болон CSV файл дээр шинжилгээ хийх боломжтой хэрэглэгчийн интерфейс.
- **Public REST API:** Гуравдагч системүүд болон хөгжүүлэгчдэд эх хэлний боловсруулалтын үйлчилгээг програмчлалын түвшинд нийлүүлэх нээлттэй программчлалын интерфейс.

Дээрх шаардлагуудыг хангах хамгийн оновчтой архитектурын шийдэл нь **Hexagonal Architecture** буюу *Ports & Adapters* загвар юм. Энэхүү загварыг сонгосноор системийн үндсэн логик нь FastAPI (backend) болон Next.js (frontend) зэрэг дэд бүтцийн технологиудаас бүрэн хамааралгүй, цэвэр бизнес логик хэлбэрээр орших боломжтой болно. Технологийн шийдлүүд нь зөвхөн ”Adapter”-аар дамжин холбогддог тул системийн үндсэн логикийг өөр өөр платформд дахин ашиглахад хялбар, уян хатан байдлыг хангадаг.

Энэхүү бүлэгт системийн функциональ ба функциональ бус шаардлагийг тодорхойлж, ерөнхий болон хэрэгжүүлэлтийн архитектурыг диаграммаар дүрслэн харуулна. Мөн системийн ажлын явцын диаграм, дарааллын диаграмм болон өгөгдлийн сангийн бүтцийг дэлгэрэнгүй танилцуулна.

2.1 Системийн шаардлага

Системийн техникийн болон хэрэглээний шаардлагуудыг тодорхойлохдоо сошиал мониторингийн системийн онцлог, Монгол хэлний боловсруулалтын хүндрэлүүд болон системийн гүйцэтгэлийг гол шалгуур болгосон.

Функциональ шаардлага

Функциональ шаардлагын хүрээнд хэрэглэгчийн гүйцэтгэх боломжтой үйлдлүүд болон системийн дотоод боловсруулалтын дамжлагуудыг Хүснэгт 2.1-д тусгав. Эдгээр шаардлагууд нь өгөгдөл цуглуулах хэсгээс эхлээд шинжилгээний үр дүнг хэрэглэгчид харуулах хүртэлх бүх процессыг хамарна.

№	Код	Нэр	Тайлбар	Хэрэглэгч	Эрэмбэ
1	F-10	Өгөгдөл оруулах	CSV файл эсвэл шууд текст оруулан NLP шинжилгээ эхлүүлэх. Нэг удаад олон мянган мөр хүлээн авах.	Шинжээч	Өндөр
2	F-20	Урьдчилан боловсруулалт	Монгол/галиг бичлэг, эможи, тусгай тэмдэгт, хасагдах үг/сул үгийг автоматаар цэвэрлэж стандартчилах.	Систем	Өндөр
3	F-30	Нэрлэсэн объект таних	Текстээс PERSON, ORG, LOC нэрлэсэн объектуудыг таньж, ангилал болон магадлалын оноотойгоор буцаах.	Шинжээч	Өндөр
4	F-40	Сэдэв загварчлал	BERTopic+KMeans болон LDA аргаар нийтлэлүүдийг сэдэвт ангилах.	Шинжээч	Өндөр
5	F-50	Хяналтын самбар (Dashboard)	Үр дүнг интерактив график дүрслэлээр харуулах.	Шинжээч	Өндөр
6	F-60	Тайлан үүсгэх	Шинжилгээний нэгтгэсэн тайланг JSON хэлбэрт API-аар буцаах.	Шинжээч	Дунд
7	F-70	программчлалын интерфейс	Гадаад систем болон хөгжүүлэгчид HTTP хүсэлтээр NLP шинжилгээний үйлчилгээг авах боломжтой байх.	Гадаад систем	Өндөр
8	F-80	Мэдлэгийн сан удирдах	Хасагдах үг/сул үг, сэдвийн ангилал, шошго нэмэх, засах, устгах.	Админ	Дунд
9	F-90	Өгөгдөл цуглуулах	Монгол мэдээний сайтаас (RSS, WordPress API, HTML scraping) нийтлэлийг цуглуулж туршилтын өгөгдлийн санд хадгалах.	Систем	Дунд

Хүснэгт 2.1: Системийн функциональ шаардлага

Функциональ бус шаардлага

Системийн тогтвортой ажиллагаа, хурд болон аюулгүй байдлыг хангах функциональ бус шаардлагуудыг Хүснэгт 2.2-д нэгтгэн харуулав. Энд техник хангамжийн нөөц ашиглалт болон алгоритмын гүйцэтгэлийн үзүүлэлтүүдийг нарийвчлан заасан болно.

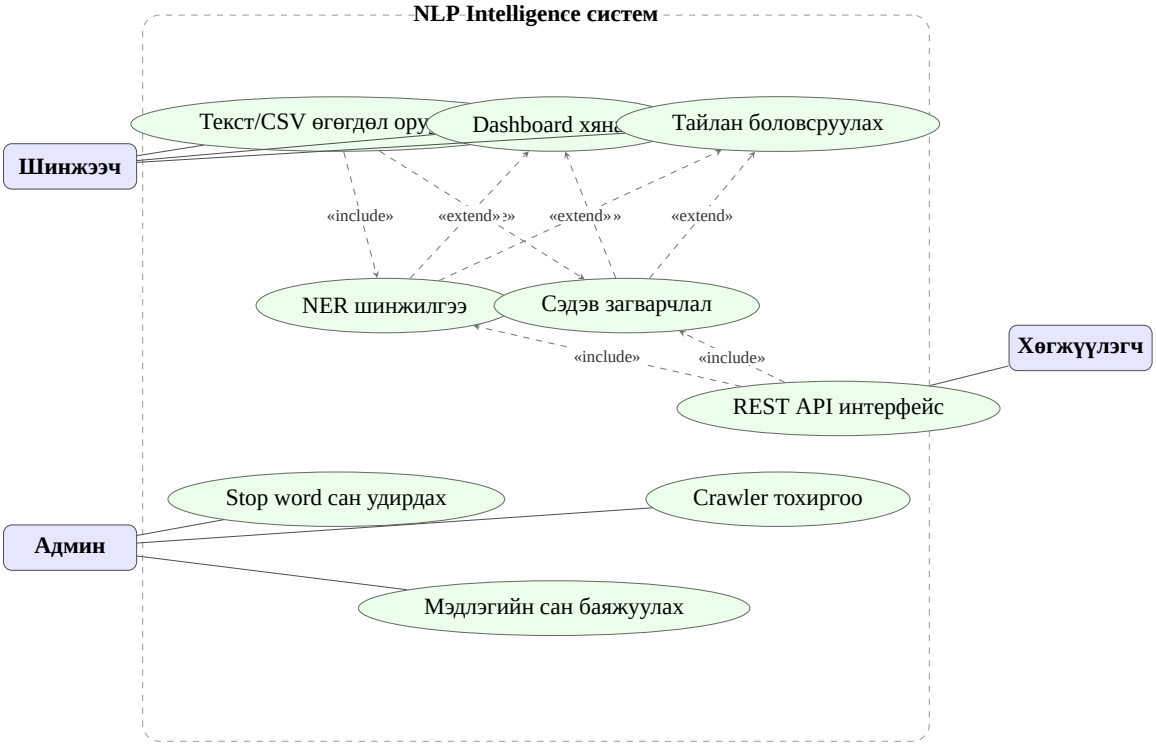
№	Код	Нэр	Тайлбар	Хэмжүүр
1	NF-10	Архитектур	Системийн үндсэн логикийг дэд бүтцээс тусгаарлах.	Хамаарал = 0
2	NF-20	Гүйцэтгэл	GPU: 20–60 ms, CPU: 300–800 ms. 1000 өгүүлбэрийн batch 2–4 сек.	Саатал $<$ 100 ms
3	NF-30	REST API	CORS зөв тохируулагдсан, JSON хариу, OpenAPI doc автоматаар үүсэх.	HTTP 200 + JSON
4	NF-40	Монгол хэл	Кирилл, галиг, ярианы хэллэг, эможи зөв боловсруулах preprocessing.	F1 $>$ 0.80
5	NF-50	Өргөтгөх боломж	Шинэ эх хэлний боловсруулалтын модуль нэмэхэд одоогийн кодыг өөрчлөхгүйгээр нэмж өргөтгөх боломжтой байх.	Өргөтгөлд нээлттэй, өөрчлөлтөд хаалттай зарчим
6	NF-60	Мэдлэгийн сан	Мэдлэгийн санг SQLite-д хадгалж, шилжүүлж, нөөцлөх боломжтой байх.	SQLite 3.x
7	NF-70	UI/UX	UI дээр 3 секунд дотор шинжилгээний үр дүн харуулах.	Хариу $<$ 3 сек
8	NF-80	Аюулгүй байдал	Оролтын өгөгдлийг шалгах, sql халдлагаас хамгаалах..	Pydantic v2

Хүснэгт 2.2: Системийн функциональ бус шаардлага

Ажлыг явцын диаграмм

Системийн функциональ бүтэц болон хэрэглэгчийн харилцан үйлчлэлийг Зураг 2.1-д үзүүлсэн **ажлын явцын** диаграммаар тодорхойлов. Систем нь *Шинжээч*, *Админ* болон

Хөгжүүлэгч гэсэн гурван үндсэн оролцогчийн үүрэгт суурилсан архитектуртай.



Зураг 2.1: Ажлын явцын диаграмм

Диаграммд дүрсэлснээр оролцогч талуудын чиг үүргийг дараах байдлаар ангилав:

- **Шинжээч:** Системийн эцсийн хэрэглэгч бөгөөд текстэн өгөгдлийг багц эсвэл задгай текст системд оруулах замаар НОТ болон сэдэв загварчлалын шинжилгээг эхлүүлнэ. Шинжилгээний үр дүнг интерактив хянах самбараар дамжуулан график дүрслэлүүдээр харж, нэгдсэн тайлан боловсруулах үүрэгтэй.
- **Админ:** Системийн тогтвортой ажиллагаа болон мэдлэгийн санг хариуцна. Үүнд хасагдах сул үгийн санг шинэчлэх, нэрлэсэн объектыг таних толь бичгийг баяжуулах, мөн цахим мэдээллийн эх сурвалжаас өгөгдөл цуглуулах тохиргоог удирдах зэрэг функцүүд багтана.
- **Хөгжүүлэгч:** Системийн байгалийн хэлний боловсруулалтын шийдлийг программчлалын интерфейсээр дамжуулан гадны системд нэгтгэх боломжтой оролцогч юм. Ингэснээр систем нь бие даасан байдлаар ажиллахаас гадна бусад платформуудад шинжилгээний үйлчилгээ үзүүлэх боломжтой.

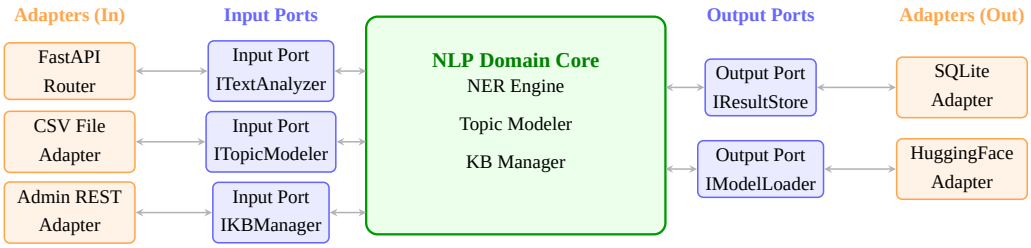
2.2 Системийн архитектур, зохиомж

Системийн ерөнхий архитектур

Системийн архитектурыг төлөвлөхдөө програм хангамжийн бүрэлдэхүүн хэсгүүдийн хамаарлыг багасгах, уян хатан байдлыг хангах зорилгоор **Hexagonal Architecture** (Ports

& Adapters) загварыг сонгон ашиглав. Энэхүү архитектур нь ”системийн бизнес логик нь дэд бүтцийн технологи, фреймворк болон гадны сангуудаас бүрэн тусгаарлагдсан байх” зарчимд суурилдаг. Системийг дараах гурван үндсэн давхаргад хувааж болно. Үүнд:

- **Домэйн логик:** Өгөгдөл боловсруулах үндсэн логик болон бизнес дүрмүүдийг агуулна. Энэхүү давхарга нь *FastAPI, SQLite* эсвэл *Next.js* зэрэг технологийн талаар ямар нэгэн мэдээлэл агуулдаггүй бөгөөд бие даан ажиллах чадвартай байна.
- **Порт:** Цөм болон бусад давхаргуудын хооронд харилцах гүүр болох интерфейсүүд (`ITextAnalyzer`, `ITopicModeler`, `IResultStore`) юм. Эдгээр нь системд ямар функцүүд хэрэгжих ёстойг тодорхойлно.
- **Адаптер:** Портуудыг бодит технологиор хэрэгжүүлэх хэсэг юм. Үүнд *FastAPI Router Adapter, SQLite Adapter, HuggingFace Model Adapter* зэрэг багтах бөгөөд тэдгээр нь домэйн цөмийн үйл ажиллагаанд нөлөөлөхгүйгээр өөрчлөгдөх эсвэл солигдох боломжтой.

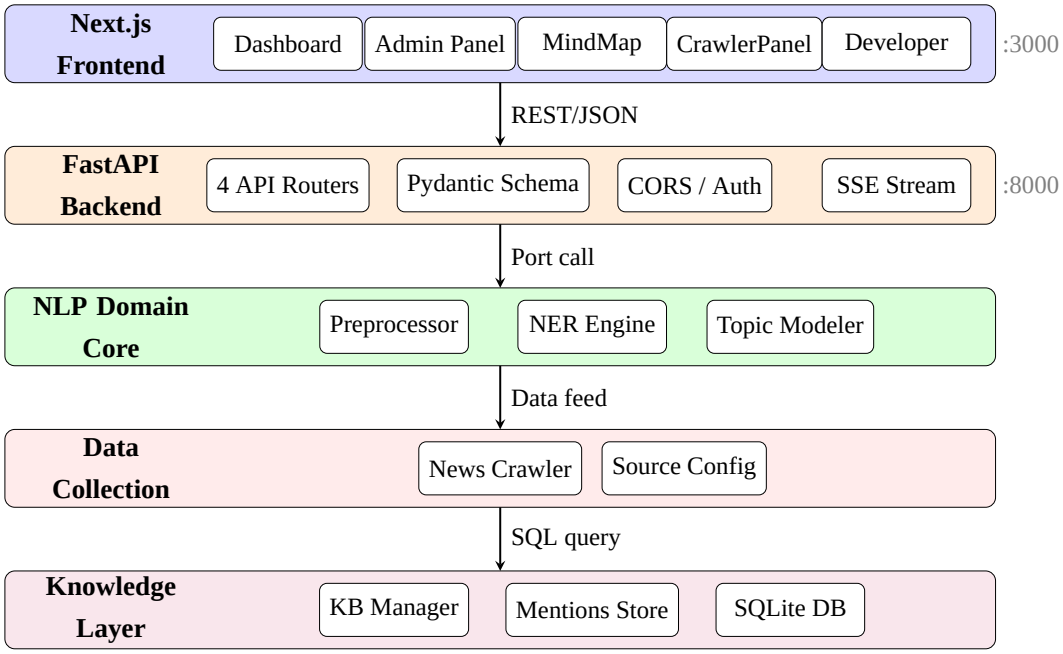


Зураг 2.2: Системийн ерөнхий архитектур

Зураг 2.2-д үзүүлснээр *Домэйн логик* нь бусад давхаргуудаас бүрэн хамааралгүй ажиллана. Зүүн талын оролтын адаптер нь оролтын портоор дамжин домайн логикийн үйл ажиллагааг идэвхжүүлдэг бол баруун талын гаралтын адаптер нь гаралтын портоор дамжин үр дүнг хадгалах, хэлний загварыг ачаалах зэрэг дэд бүтцийн ажлуудыг гүйцэтгэнэ. Ийм архитектур ашигласнаар системийн бизнес логикийг өөрчлөхгүйгээр өгөгдлийн сан эсвэл вэб фреймворкийг шинэчлэх боломжтой төдийгүй, нэгэн ижил логикийг вэб аппликейшн болон нээлттэй программчлалын интерфейс зэрэг олон төрлийн интерфейст дахин ашиглах давуу талыг бүрдүүлж байна.

Системийн зохиомж

Судалгааны үр дүнг бодит орчинд турших, үнэлэх зорилгоор боловсруулсан систем нь таван үндсэн технологийн давхаргаас бүрдсэн архитектуртай (Зураг 2.3).



Зураг 2.3: Системийн давхаргат архитектур

Уг архитектур нь модульчлагдсан бүтэцтэй бөгөөд давхарга бүр дараах үүргийг гүйцэтгэнэ. Үүнд:

- **Хэрэглэгчийн интерфейс:** Хэрэглэгчид шинжилгээний үр дүнг дашбоард болон оюуны зураглал хэлбэрээр харуулах, мэдээ цуглуулах, удирдах боломжтой интерфейсээр хангана.
- **Серверийн талын логик:** Гадаад системүүдтэй REST API-аар харилцаж, үндсэн логик рүү хүсэлтийг дамжуулах гарц болно.
- **Домэйн логик:** Текстэд урьдчилсан боловсруулалт хийх, нэрлэсэн объект таних болон сэдэв загварчлалын үндсэн логик энд төвлөрнө.
- **Өгөгдөл цуглуулах, хадгалах хэсэг** Мэдээний эх сурвалжуудаас өгөгдөл цуглуулж, SQLite өгөгдлийн санд бүтэцлэгдсэн хэлбэрээр хадгална.

REST API эндпойнт

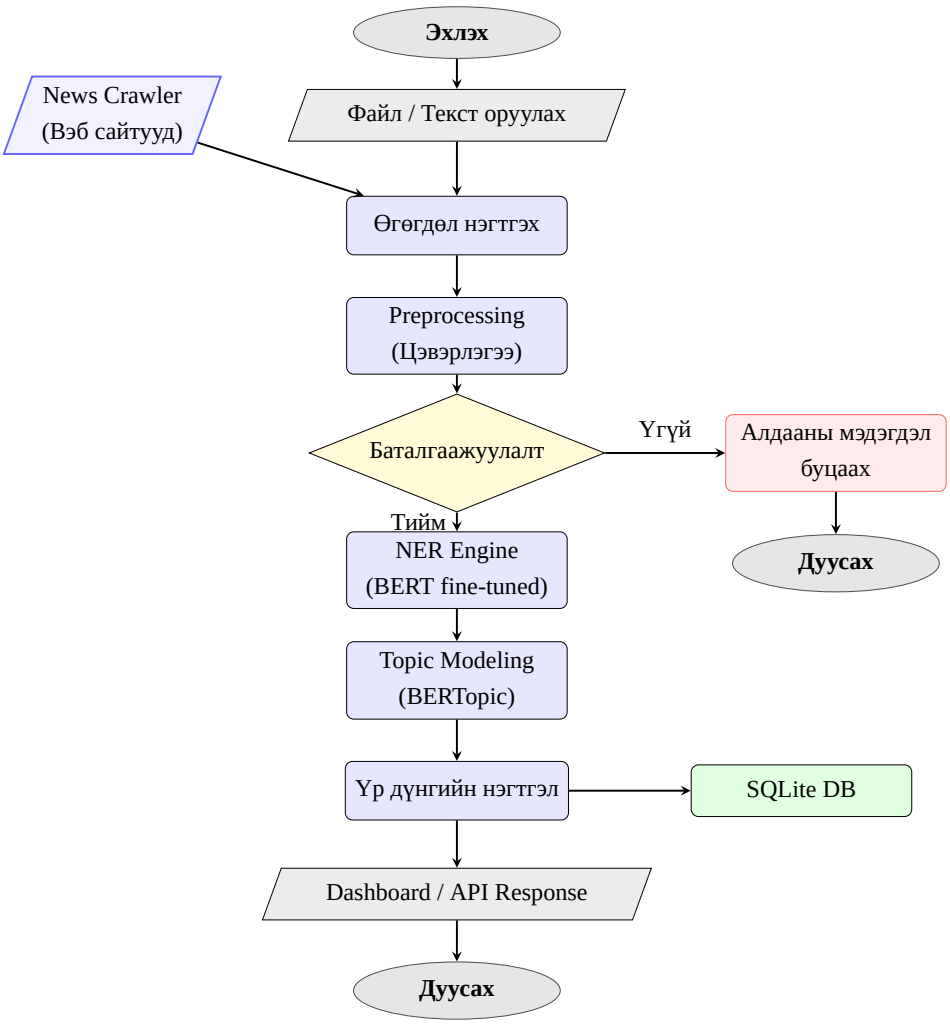
Системийн функцуудыг гадаад систем болон хөгжүүлэгчид ашиглах боломжийг FastAPI Gateway-д суурилсан REST API хангана. Нийт эндпойнтуудыг үйл ажиллагааны чиглэлээр нь ангилан дараах хүснэгтэд тодорхойлов (Хүснэгт 2.3).

Метод	Эндпойнт	Оролт (Input)	Гаралт (Output)
Шинжилгээний модулийн эндпойнтууд			
POST	`/analyze/ner`	`{texts: [str]}`	Нэрлэсэн нэгж, байршил
POST	`/analyze/topics`	`{texts, method}`	Сэдэв, түлхүүр үгс
POST	`/analyze/full`	`{texts: [str]}`	Цогц шинжилгээ (NER+Topic)
Хяналтын самбар ба Мэдээллийн эндпойнтууд			
GET	`/dashboard/stats`	query params	KPI үзүүлэлтүүд, өсөлт
GET	`/dashboard/timeline`	date range	Хугацааны хамаарал
GET	`/dashboard/mentions`	filters, page	Шүүсэн илэрцүүд
GET	`/dashboard/top-entities`	—	Топ PER, ORG, LOC
Удирдах хэсэг ба Crawler			
POST	`/crawler/crawl`	—	Өгөгдөл цуглуулалт эхлүүлэх
GET	`/admin/knowledge`	—	Мэдлэгийн сангийн бүртгэл

Хүснэгт 2.3: REST API-ийн үндсэн эндпойнтуудын тодорхойлолт

Байгалийн хэлний боловсруулалтын шийдлийн мэдээллийн урсгал

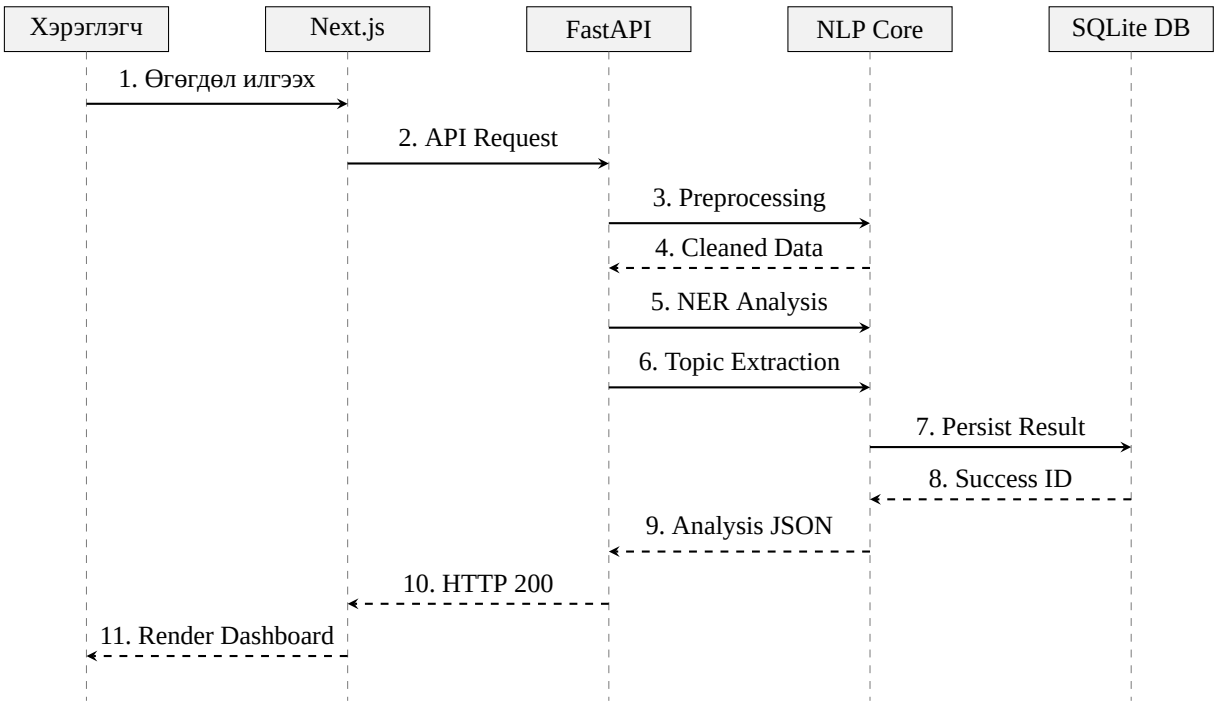
Системийн өгөгдөл боловсруулах ерөнхий логик урсгал нь зураг 2.4-д үзүүлсэн дарааллаар явагдана. Өгөгдөл нь хэрэглэгчийн оруулсан текст, файл болон вэб crawler гэсэн хоёр үндсэн сувгаар системд орж ирэх бөгөөд урьдчилан боловсруулах шатанд цэвэрлэгдэж, ”NER Engine” болон ”Topic Modeling” модулиудаар дамжин боловсруулагдаж өгөгдлийн сан руу хадгалагдана.



Зураг 2.4: Байгалийн хэлний боловсруулалтын шийдлийн мэдээллийн урсгал

Бизнесийн процессыг урсгалын диаграмм

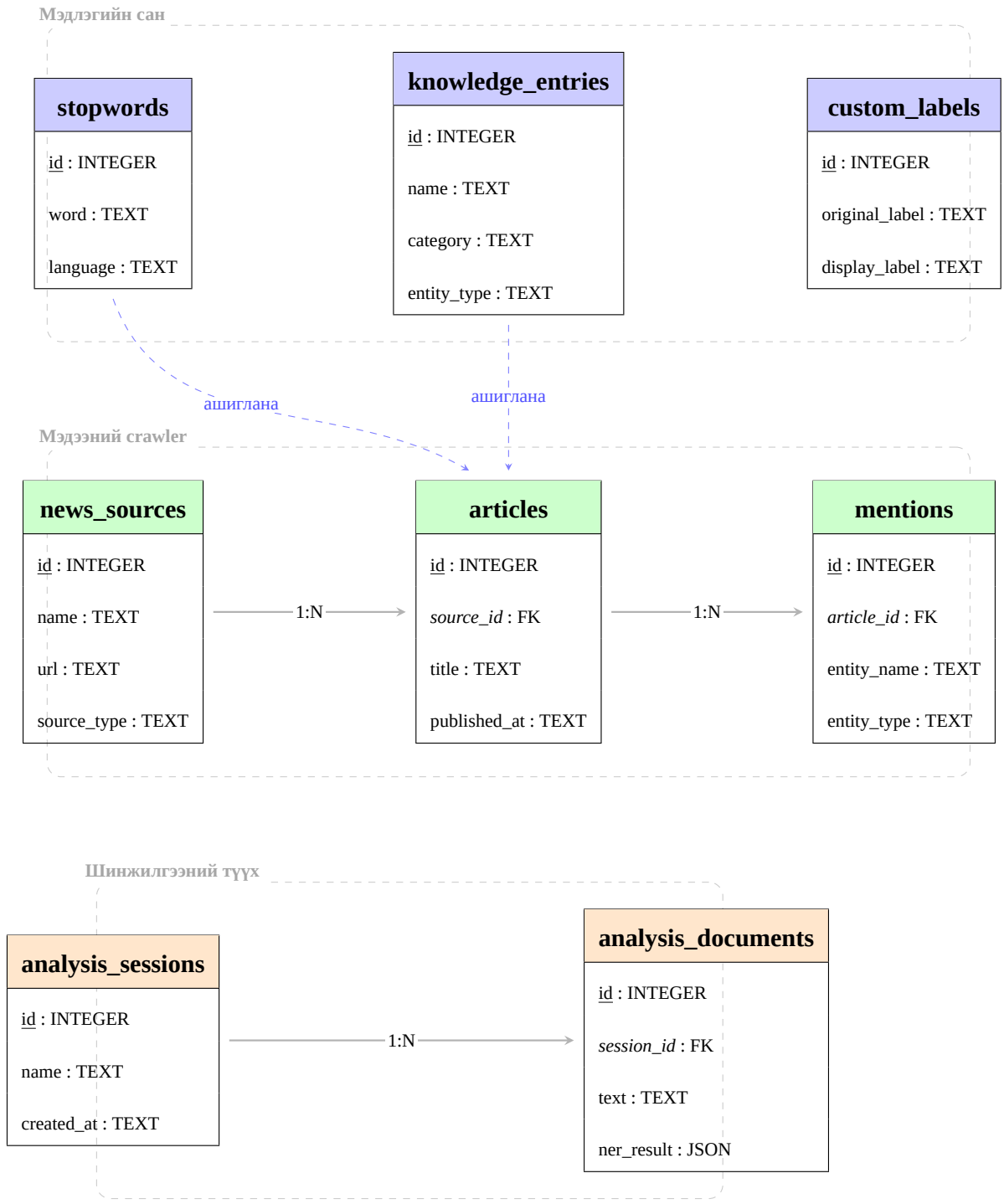
Хэрэглэгчийн хүсэлт систем дотор хэрхэн дамжихыг ”POST /analyze/full” жишээн дээр зураг 2.5-д харуулав. Энэхүү диаграмм нь фронтэнд, бэкэнд болон байгалийн хэлний боловсруулалтын цөм хэсгийн харилцан үйлчлэлийг цаг хугацааны дарааллаар илэрхийлнэ.



Зураг 2.5: Системийн бүрэлдэхүүн хэсгүүдийн харилцан үйлчлэлийн дараалал

2.3 Өгөгдлийн сангийн зохиомж

Системийн мэдээллийн тогтвортой байдлыг хангах зорилгоор **SQLite** өгөгдлийн санг ашигласан. Энэ нь хөнгөн бүтэцтэй, файл хэлбэрээр хадгалагддаг тул дипломын ажлын хүрээнд туршилт болон нүүлгэн шилжүүлэлт хийхэд нэн тохиромжтой юм. Өгөгдлийн сангийн логик бүтцийг (ERD) зураг 2.6-д үзүүлэв.



Зураг 2.6: Өгөгдлийн сангийн ER диаграмм

Нийт 8 хүснэгт нь **нэг SQLite өгөгдлийн санд** хадгалагддаг — энэ нь 3 тусдаа өгөгдлийн сан биш. Диаграмм дахь гурван хайрцаг нь зөвхөн *логик бүлэглэлт* бөгөөд хүснэгтүүдийн үүрэг, харилцааг ойлгомжтой харуулах зорилгоор ийнхүү бүлэглэсэн болно.

Мэдлэгийн сан бүлэг (`stopwords`, `knowledge_entries`, `custom_labels`) нь бусад хүснэгтүүдтэй гадаад түлхүүрийн холболтгүй байдаг — энэ нь зориудын шийдэл. Эдгээр хүснэгтийг NLP ажлын урсгал программ хангамжийн түвшинд шууд дуудан ашигладаг тул мэдлэгийн санг бусад бүлгээс бие даан засварлах, шинэчлэх боломжтой

байна. Тухайлбал `stopwords` нь preprocessing үе шатанд, `knowledge_entries` нь NER шинжилгээний үед лавлагааны сан болдог.

Мэдээний crawler бүлэг нь `news_sources` $\xrightarrow{1:N}$ `articles` $\xrightarrow{1:N}$ `mentions` гэсэн гинжин холболттой. Нэг эх сурвалж олон нийтлэлтэй, нэг нийтлэл олон нэрлэсэн объект дурдалттай байж болно. `articles.source_id` болон `mentions.article_id` нь гадаад түлхүүр болно.

Шинжилгээний түүх бүлэг нь `analysis_sessions` $\xrightarrow{1:N}$ `analysis_documents` холболттой бөгөөд crawler бүлгээс тусдаа ажилладаг. Энэ бүлэг нь зөвхөн CSV файлаар оруулсан гарын авлагат шинжилгээний үр дүнг хадгалах зориулалттай.

Өгөгдлийн санд нийт 8 хүснэгт байна. Хүснэгт 2.4-д тус бүрийн гол талбарууд болон зориулалтыг нэгтгэн харуулав.

Хүснэгт	**Гол талбарууд**	**Зориулалт**
`stopwords`	id, word, language	Монгол/Англи хасагдах үг болон сул үг. Preprocessing-д ашиглагдана.
`knowledge_entries`	id, name, category, entity_type, synonyms	Админ бүртгэсэн нэрлэсэн объектуудын сан.
`custom_labels`	id, original_label, display_label	NER шошгоны дүрслэлийн тохиргоо (жишээ: PER → “Улс төрч”).
`analysis_sessions`	id, name, created_at	CSV оруулсан шинжилгээний түүх.
`analysis_documents`	id, session_id, text, result_json	Шинжилсэн документ бүрийн үр дүн.
`news_sources`	id, name, url, source_type	11 Монгол мэдээний сайтын жагсаалт (RSS, WP API, HTML).
`articles`	id, title, content, source, date, image_url	Crawler-ийн цуглуулсан мэдээний нийтлэлүүд.
`mentions`	id, article_id, entity_name, entity_type, topic, confidence	Нийтлэлийн NLP шинжилгээний үр дүн (HOT болон сэдэв)-ийг хадгалах хүснэгт.

Хүснэгт 2.4: Өгөгдлийн сангийн хүснэгтүүдийн бүтэц ба зориулалт

2.4 Байгалийн хэлний боловсруулалтын загваруудын сонголт

Системийн үндсэн логик болох байгалийн хэл боловсруулалтын модулиудад хэрэглэх технологийг сонгохдоо Монгол хэлний залгаварт бүтэц, семантик онцлог болон бодит цагийн гүйцэтгэлийн шаардлагыг гол шалгуур болгов. Тодорхой загвар сонгохоос өмнө дараах ерөнхий гурван хандлагыг харьцуулан үнэлсэн бөгөөд тэдгээрийн дагуу дараах дэд бүлгүүдэд НОТ болон сэдэв загварчлалд ашиглагдах тус бүрийн шийдлүүдийг мөн туршилт хийх замаар сонгосон болно.

БХБ-ын хандлагын ерөнхий харьцуулалт

Текст боловсруулах ашиглагддаг гурван үндсэн хандлагыг тооцооллын нөөц, боловсруулалтын хурд болон Монгол хэлний тусгай дэмжлэгийн шалгууруар харьцуулан дараах байдлаар үнэлэв.

1. **Дүрэм, мэдлэгийн санд суурилсан арга:** Домайны тусгай нэр томьёо болон нэрлэсэн объектуудыг урьдчилан бэлтгэсэн мэдлэгийн сантай шууд харьцуулан ангилдаг арга юм. Тооцооллын нөөц маш бага шаарддаг (1–5 мс) бөгөөд мэдлэгийн санд бүртгэгдсэн нэршлийг нарийвчлалтай таньдаг давуу талтай. Гэвч урьдчилан тодорхойлогдоогүй шинэ нэршил болон тархалтын нөхцөлд зохицох уян хатан чанар дутмаг тул хурдацтай өөрчлөгдөж буй сошиал өгөгдөлд хязгаарлагдмал.

2. **Нарийвчлан сургасан BERT загвар:** Transformer архитектурт суурилсан олон хэлний урьдчилан сургасан загварыг тухайлсан даалгаварт зохицуулан дахин сургах арга юм. T4 GPU орчинд нэг өгүүлбэрийг 20–60 мс-д боловсруулах чадвартай бөгөөд дэд үгийн токенжуулалт ашигладаг тул Монгол хэлний залгаварт бүтцийн асуудлыг оновчтой шийдвэрлэдэг. Зардал болон нарийвчлалын хооронд хамгийн тэнцвэртэй шийдлийг өгдөг.

3. **Том хэлний загварын программчлалын интерфейс:** GPT-4, Claude зэрэг бэлэн загваруудыг API-аар дамжуулан ашиглах хандлага юм. Өндөр нарийвчлалтай боловч 100,000 текст боловсруулахад $200–$300 зардалтай бөгөөд сүлжээний хамааралтай тул бодит цагийн горимд 1–3 секундын хоцрогдол үүсгэдэг.

Үнэлгээний шалгуур	Мэдлэгийн сан	Fine-tuned BERT	LLM API
Тооцооллын нөөц (GPU)	Шаардлагагүй	Зөвлөмж болгох	Шаардлагагүй
Боловсруулалтын хурд	1–5 мс	20–100 мс	1–3 сек
100K текстийн зардал	Маш бага	Бага	Өндөр
Семантик нарийвчлал	Дунд	Өндөр	Маш өндөр
Сүлжээний хамаарал	Офлайн	Офлайн	Онлайн
Уян хатан байдал	Сул	Дунд	Маш өндөр
Монгол хэлний дэмжлэг	Өндөр (Гараар)	Өндөр (Fine-tune)	Дунд

Хүснэгт 2.5: Текст боловсруулах хандлагуудын харьцуулалт

Хүснэгт 2.5-д үзүүлсэн харьцуулалтад үндэслэн энэхүү системд **нарийвчлан сургасан BERT загвар болон Мэдлэгийн санг хослуулсан шийдлийг** сонгов. Монгол хэлний тэмдэглэгээт корпус хомс байгаа нөхцөлд олон хэлний BERT загварыг нарийвчлан сургах нь семантик нарийвчлал болон хурдны тэнцвэрийг хангадаг. Мэдлэгийн сан нь домайн бүрийн тусгай нэршлийг нарийвчлан таних нөхөн чадавхийг бий болгож, хоёр арга хоорондоо нэмэлт байдлаар ажиллана. LLM API-г сонгоогүй гол шалтгаан нь санхүүгийн өндөр зардал болон сүлжээний хамааралтай байдал нь бодит цагийн систем барих шаардлагад нийцэхгүй гэж үнэлсэнтэй холбоотой.

Нэрлэсэн объект таних загварын харьцуулалт

Системийн NER модульд зориулан Монгол хэл дээрх мэдээний текстээс хүн (PER), байгууллага (ORG), байршил (LOC) болон бусад (MISC) нэрлэсэн объектуудыг ялгах даалгаварт хамгийн тохиромжтой загварыг сонгохын тулд хэд хэдэн олон хэлний загваруудыг харьцуулан туршив.

NER модульд зориулан олон хэлний гурван загварыг Монгол хэлний баримт бичгийн дэмжлэг болон архитектурын тохиромжийн үүднээс харьцуулан үнэллээ. Хүснэгт 2.6-д загвар бүрийн гол шинж чанарыг нэгтгэн харуулав.

Загвар	Монгол хэлний дэмжлэг	Саатал (CPU)	Архитектурын онцлог
`Davlan/bert-multilingual`	Сайн (Subword түвшинд)	∼450ms	Алтай язгуурын хэлнүүд дэмждэг, залгаварт хэлэнд тохиромжтой
`spaCy xx_ent_wiki_sm`	Дэмжихгүй	12ms	Монгол хэл дэмжихгүй (баримт бичигт заасан)
`flair/ner-multi`	Хязгаарлагдмал	∼550ms	Монгол NER корпусгүйгээр сургагдсан

Хүснэгт 2.6: NER загваруудын харьцуулалт

Сонголтын үндэслэл: `spaCy xx_ent_wiki_sm` загвар нь Монгол хэлийг дэмждэггүй болохыг баримт бичигт тодорхой заасан тул анхнаасаа хасагдав. `flair/ner-multi` нь олон хэлний дэмжлэгтэй ч Монгол NER-д зориулсан тэмдэглэгдсэн корпусгүйгээр сургагдсан бөгөөд залгаварт үгийн бүтцэд тулгарах токенжуулалтын асуудлыг шийдэх механизмгүй.

`Davlan/bert-base-multilingual-cased-ner-hrl` загварыг сонгосон гол шалтгаан нь гурав байна. Нэгдүгээрт, загвар нь 10 гаруй хэл дээр сургагдсан бөгөөд тэдгээрт Монголтой ижил залгаварт морфологийн бүтэцтэй алтай язгуурын хэлнүүд (Турк, Казак г.м) орсон байдаг. Хоёрдугаарт, BERT-ийн дэд үгийн токенжуулалт нь Монгол хэлний урт залгаварт хэлбэрүүдийг (“Улаанбаатарт”, “Монголынхоо” г.м) утга нь хадгалагдах байдлаар задалдаг. Гуравдугаарт, HRL (High Resource Languages) сургалтын аргачлал нь нөөц хомс хэлнүүдийг дэмжихэд тохиромжтой болохыг холбогдох судалгаа [5] батална. Уг загварт нарийн тааруулга хийж Монгол хэлний NER-ийн нарийвчлалыг нэмэгдүүлэх арга зүйн дэлгэрэнгүйг Гуравдугаар бүлэгт тайлбарласан болно.

Сэдэв загварчлалын арга зүй

Монгол хэл дээрх нийгмийн сүлжээний болон мэдээний их хэмжээний текстээс далд сэдвүүдийг автоматаар илрүүлэхийн тулд бид уламжлалт статистик аргуудын оронд гүнзгий сургалт болон кластерчлалын хосолсон (Hybrid) хоёр аргыг зэрэгцүүлэн хэрэгжүүлэхээр шийдвэрлэснийг энд тайлбарлая:

1. **BERTopic + KMeans кластерчлал:** Урьдчилан тааварлах боломжгүй, цаг үеийн шинэ сэдвүүдийг автоматаар илрүүлэх зорилгоор *BERTopic* хүрээг ашиглав. Текстийг семантик вектор буюу embedding болгохдоо 50 гаруй хэлний мэдлэгтэй `paraphrase-multilingual-mpnet-base-v2` загварыг ашигласан нь Монгол үгсийн салаа утгыг бүрэн салгаж ойлгох боломжийг олгосон юм. Мөн BERTopic-ийн үндсэн HDBSCAN кластерчлал нь жижиг хэмжээний датасет дээр ихэнх

текстийг гажиг (outlier) болгон харьцдаг сул талтай тул түүний оронд бага болон их хэмжээний (*mini and large batches*) өгөгдөл дээр тогтвортой ажиллах *KMeans* алгоритмыг хэрэглэж, өгөгдлийн хэмжээнээс хамаарч сэдвийн тоог ($K = 5 \sim 15$ хооронд) динамикаар тогтоох адаптив механизмыг зохиомжлов. Үүгээр зогсохгүй, Монгол хэлний залгаварт онцлогт зориулсан специал *tokenizer* хөгжүүлж (цагийн болон харьяалах тэмдэглэгч (*-ийн, -г*) устгах буюу *suffix stripping*), тусгай *stopword*-үүдийг c-TF-IDF шатанд нэмж шүүснээр сэдвийн түлхүүр үгс цэвэр, оновчтой гарах боломж бүрдсэн.

2. **Zero-shot Classification:** Хэрэглэгчээс урьдчилан өгсөн тогтмол категориудад (жишээ нь: “Улс төр”, “Эрүүл мэнд”, “Санхүү”) текстийг хурдан ангилах хэрэгцээнд зориулж энэ аргыг үнэлгээнд нэмэлтээр ашиглав. Ингэснээр систем нь бие даан шинэ сэдэв нээхийн зэрэгцээ, бизнесийн хэрэгцээ шаардлагад тулгуурласан чиглэл рүү хуваарилах боломжтой болно.

Системийн зохиомжийн дүгнэлт

Энэ бүлэгт сошиал мониторингийн системийн үйл ажиллагааны шаардлагыг шинжилж, архитектур болон дэд бүтцийг **Hexagonal Design** (Порт болон Адаптер) зарчмаар зохиомжиллоо. Энэхүү архитектур нь өгөгдөл боловсруулах бизнесийн логикийг интерфейс болон гадаад технологиос бүрэн тусгаарлаж, системийн уян хатан, модуляр байдлыг хангаж байна. Түүнчлэн хэрэглэгчийн текст болон веб crawler-аар цугларсан өгөгдөл нь урьдчилсан боловсруулалт, НОТ болон сэдэв загварчлалын үе шатуудад дамжих өгөгдлийн урсгалыг нарийвчлан тодорхойлов. Өмнөх бүлгийн онолын үндэслэлд тулгуурлан Монгол хэлний залгаварт бүтэц, онцлогт нийцүүлэн НОТ даалгаварт олон хэлний BERT загварыг *character-offset* аргачлалаар нарийвчлан сургах, харин сэдэв загварчлалд BERTopic болон KMeans-ийг хослуулан ашиглах технологийн оновчтой шийдлийг практик туршилтаар баталгаажуулав. Системийн энэхүү цогц зохиомж нь өндөр нарийвчлалтайгаар бодит цагт ажиллах бат бөх суурийг тавьсан бөгөөд эдгээр зохиомжийн бодит кодын хэрэгжүүлэлт, туршилтын үр дүнг дараагийн бүлэгт нотолгоотойгоор авч үзнэ.

Бүлэг 3

Хэрэгжүүлэлт, туршилт

Энэхүү бүлэгт өмнөх бүлгүүдэд тодорхойлсон системийн архитектур болон сонгосон загваруудын бодит кодын хэрэгжүүлэлтийг танилцуулна. Хөгжүүлэлтийн явцыг өгөгдлийн урсгалын логик дарааллаар буюу мэдээлэл цуглуулахаас эхлүүлээд эцсийн хэрэглэгчийн интерфейс хүртэл шат дараалан авч үзнэ.

3.1 Ашигласан технологиуд

Backend технологиуд

Технологи	Хувилбар	Зориулалт
Python	3.11	Үндсэн хэл — NLP core болон API backend
FastAPI	0.110	Async REST API framework, OpenAPI автомат
HuggingFace Transformers	4.40	BERT НОТ pipeline болон нарийн тааруулга
BERTopic	0.16	Sentence вектор дүрслэл + HDBSCAN/KMeans кластер
sentence-transformers	2.7	Multilingual sentence вектор дүрслэл (paraphrase-mpnet)
SQLite / sqlite3	3.45	Knowledge Base өгөгдлийн сан
Pydantic	2.7	API schema validation, dataclass
Docker	25	Container-д суурилсан байршуулалт

Хүснэгт 3.1: Backend технологиуд

Frontend технологиуд

Технологи	Хувилбар	Зориулалт
Next.js	14 (App Router)	React-д суурилсан SSR/CSR framework
Tailwind CSS	3.4	Utility-first CSS, glassmorphic dark-mode
Recharts	2.12	Pie chart, Bar chart, Timeline дүрслэл
TypeScript	5.4	Статик тип шалгалт

Хүснэгт 3.2: Frontend технологиуд

3.2 Өгөгдөл цуглуулалт ба Crawler модулийн хэрэгжүүлэлт

Системийн өгөгдлийн цуглуулалтын нэг чухал хэсэг нь мэдээний сайтууд болон сошиал медиа платформуудаас автоматаар нийтлэл татах нэгдсэн модуль юм. Энэхүү модуль нь мэдээллийн төрлөөс хамааран өөр өөр дэд хэсгүүдэд хуваагддаг.

Мэдээний сайтуудаас өгөгдөл татах

Монгол мэдээний сайтуудаас автоматаар нийтлэл цуглуулдаг `NewsCrawler` модуль нь нийт 11 Монгол мэдээний сайтыг гурван аргаар дэмжин ажилладаг:

Арга	Сайтууд	Тайлбар
RSS	ikon.mn	`feedparser` ашиглан RSS feed-ээс нийтлэл татах
WordPress API	eguur.mn	WP REST API (`/wp-json/wp/v2/posts`) ашиглах
HTML Scraping	gogo.mn, news.mn, montsame.mn, zarig.mn, eagle.mn, peak.mn, ergelt.mn, sonin.mn, ubn.mn	Сайт бүрийн бүтцэд тохирсон HTML parser

Хүснэгт 3.3: Мэдээний crawler-ийн дэмждэг сайтууд ба аргууд

Crawler нь ангилал/нүүр хуудаснаас нийтлэлийн холбоосуудыг илрүүлж, тус бүрийн гарчиг, агуулга, огноо, `og:image` зургийг задалдаг. 50 тэмдэгтээс бага агуулгатай нийтлэлийг автоматаар хасдаг. Зургуудыг `media/` хавтаст MD5 хэшлэсэн нэрээр татаж хадгалдаг. Мөн танигдаагүй сайтуудад зориулсан generic fallback scraper-тай.

```
class NewsCrawler:
    def crawl_source(
        self, source: Dict,
        date_from: Optional[str] = None,
```

```
        date_to: Optional[str] = None,
        max_articles: int = 200,
    ) -> List[Dict]:
        source_type = source.get("source_type", "rss")
        if source_type == "rss":
            return self._crawl_rss(
                source["id"], source.get("rss_url", ""),
                date_from, date_to, max_articles)
        elif source_type == "wp_api":
            articles = self._crawl_wp_api(
                source["id"], source.get("rss_url", ""),
                date_from, date_to, max_articles)
            if not articles:  # WP API амжилтгүйбол HTMLд- шилжих
                base_url = source.get("url", "")
                domain = self._get_domain(base_url)
                scraper = self._html_scrapers.get(
                    domain, self._scrape_generic)
                articles = scraper(source["id"], base_url,
                                   date_from, date_to, max_articles)
            return articles
        else:
            base_url = source.get("url", "")
            domain = self._get_domain(base_url)
            scraper = self._html_scrapers.get(
                domain, self._scrape_generic)
            return scraper(source["id"], base_url,
                           date_from, date_to, max_articles)
```

Listing 3.1: NewsCrawler — crawl_source() үндсэн логик

Сошиал медиагаас өгөгдөл татах болон тулгарсан сорилтууд

Мэдээний сайтаас гадна олон нийтийн сүлжээнээс Монгол хэл дээрх өгөгдлийг цаг алдалгүй татах нь системийн гол зорилгын нэг юм. Энэхүү шаардлагыг хангахын тулд `social_scraper.py` дэлгэврийн модулийг хөгжүүлж, Instagram, X (Twitter), болон Facebook платформуудаас мэдээлэл цуглуулах `scrape_platform` нэгдсэн функцийг хэрэгжүүлэв.

Гэвч сошиал платформуудаас автоматаар өгөгдөл татах явцад тухайн байгууллагуудын аюулгүй байдлын бодлогоос үүдсэн томоохон техникийн сорилтуудтай тулгарсан бөгөөд тэдгээрийг дараах байдлаар шийдвэрлэсэн:

1. **Instagram (Хязгаарлалт ба 403 алдаа):** Анх `instaloader` сан ашиглан хаштаг эсвэл түлхүүр үгээр мэдээлэл татахаар хэрэгжүүлсэн боловч Instagram-ийн

серверээс хандалтыг маш хурдан хугацаанд хааж (403 Forbidden Error) байв. Энэхүү асуудлыг давахын тулд жинхэнэ хэрэглэгчийн веб хөтөчийг дуурайх `Selenium fallback` (Селениум нөөц арга) механизмаар дамжуулан хөтчөөс хайлт хийж, нийтлэлийн зураг болон сэтгэгдлийн хамт татах шийдэл нэмж оруулсан.

2. **X буюу Twitter (Хандалтын хатуу хязгаарлалт):** Twitter нь хуучин нээлттэй API-аа хаасан тул `twikit` санг ашиглаж эхэлсэн. Гэвч хандалтын хязгаарлалт болон заавал нэвтрэх шаардлага тулгарсан бөгөөд үүнийг `cookies.json` файлд хэрэглэгчийн күүкиг хадгалах болон `async` (асинхрон) горимоор таталтыг зэрэг гүйцэтгэх замаар оновчтой болгосон. Хэрэв шууд хандалт бүтэлгүйтвэл мөн адил үл үзэгдэх цонхоор ажиллах хөтчийн Селениум горим руу автоматаар шилждэг.

3. **Facebook (Нэвтрэлтийн блок):** Хаалттай групп болон хуудсуудаас дата авах үед `facebook-scraper` сан нь ихэвчлэн түр зуурын эсвэл бүр мөсөн аккаунт түгжигдэх эрсдэл дагуулдаг тул зөвхөн олон нийтэд нээлттэй хуудсууд дээр хязгаартай хандах тохиргоо хийгдсэн.

Эдгээр сорилтуудад үндэслэн тухайн платформ тус бүрийн сангууд (instaloader, twikit) амжилтгүй болсон үед автоматаар Selenium-ийн бодит хөтөч ажиллуулан хуудсыг уншиж мэдээллийг задалдаг хоёр шатлалт архитектур бичигдсэн нь системийн найдвартай ажиллагааг маш өндөр түвшинд хангаж чадсан юм.

3.3 Өгөгдлийн урьдчилсан боловсруулалт

Цуглуулсан өгөгдөл нь шууд загварт оруулахад тохиромжгүй, дуу чимээ буюу хэрэгцээгүй тэмдэгтүүд ихтэй байдаг тул эхлээд цэвэрлэгээ буюу урьдчилсан боловсруулалт хийх шаардлагатай.

Хоёр горимт preprocessing архитектур

Preprocessing нь NLP pipeline-ийн эхний үе шат бөгөөд оролтын текст НОТ болон Сэдэв загварчлал модулиуд руу орохоос өмнө гүйцэтгэгддэг (зураг 2.4). Туршилтын явцад нэг preprocessing горим хоёр модульд хангалтгүй болохыг тодорхойлсон тул зориулалтаар нь ялгаатай хоёр горим хэрэгжүүллээ.

preprocess_nlp() горим нь НОТ-д зориулагдсан. BERT загвар текстийн бүтэц болон том-жижиг үсгийг мэдрэмтгий байдлаар ашигладаг тул энэ горим цэг тэмдэг болон том үсгийг хадгалж, нэрийн хэлбэрийг (жишээ нь `б.амар` → `Б.Амар`) сэргээдэг.

preprocess_tm() горим нь BERTopic-д зориулагдсан. Сэдэв загварчлалд зөвхөн агуулга бүхий үгс чухал тул Кирилл/Latin үсэг болон тооноос бусдыг хасаж, жижиг үсэгт хувиргаж, хасагдах үг/сул үг-ийг устгах гүнзгий цэвэрлэгээ хийдэг.

```
1 class Preprocessor:
2     def preprocess_nlp(self, text: str) -> str:
```

```
        """НОТСентимент+: нэрийнбичиглэл , томүсэгхадгалахгорим    .
           """
        if not isinstance(text, str):
            return ""
        # convert_emoji=True: ▫→[ANGRY] маркер BERTд- очно
        text = self.clean_basic(text, replace_url=True,
                                convert_emoji=True)
        text = _capitalize_for_ner(text)   # бамар.  -> БАмар .
        text = _restore_names(text)        # БАмар_  -> БАмар .
        return text

    def preprocess_tm(self, text: str) -> str:
        """BERTopic: үгийнүндэсхүртэлхгүнзгийцэвэрлэгээнийгорим     .
           """
        if not isinstance(text, str):
            return ""
        text = self.clean_basic(text, replace_url=False)
        text = self.clean_deep(text)       # зөвхөнКирилл /Latinтоо/
        text = text.lower()
        text = self._remove_stopwords(text)
        text = re.sub(r"\bаяөүё[-]_аяөүё([-a-zаяөүё]+(?:-[-a-z]+)*)\b
           ",
            r"\1", text)  # АБат_ →батэхний  ( үсэг TMд- хэрэггүй)
        return re.sub(r"\s+", " ", text).strip()

    def preprocess_dual(self, text: str) -> Tuple[str, str]:
        """Нэг дуудлагаархоёргоримынүрдүнгбуцаана     ."""
        return self.preprocess_nlp(text), self.preprocess_tm(text)
```

Listing 3.2: Preprocessing хоёр горим — preprocess_dual()

Монгол нэр хамгаалах механизм

Монгол хэлэнд хүний нэрийг ихэвчлэн овгийн эхний үсэг болон нэрийн хослолоор тэмдэглэдэг бөгөөд “А.Бат” хэлбэр нийтлэг хэрэглэгддэг заншил юм. Preprocessing дуусахад цэг нь арилж “А Бат” болдог тул BERT загвар нэрийг бүтнээр таньдаггүй болж байсан. Энэ асуудлыг нэрийг “А_Бат” хэлбэрт хамгаалж, дараа нь сэргээх механизмаар шийдлээ:

```
# Дээдүсгийнэхнийүсэг   : АБатЭрдэнэ.- →АБат _Эрдэнэ-
MN_NAME_UPPER = re.compile(
    r"\bАЯӨҮЁ([-])\.\s*"
    r"АЯаяӨөҮүЁёаяөүё([--][-a-z]+"
    r"АЯаяӨөҮүЁёаяөүё(?:-[--][-a-z]+)*)"
```

```
)
# Жижигүсгийнэхнийүсэг   : бамар. →БАмар _ сошиал ( медиадтүгээмэл )
MN_NAME_LOWER = re.compile(
    r"\bаяөүё([-])\.\s*"
    r"аяөүё([-a-z]+"
    r"аяөүё(?:-[-a-z]+)*)"
)
```

Listing 3.3: Нэр хамгаалах regex — дээд болон жижиг үсэг хоёуланд

Туршилтын үр дүн: Нэр хамгаалах механизм нэмэхээс өмнө “б.амар улс төрч” текстэд NOT ямар ч PER объект буцааж байгаагүй. Нэмсний дараа “Б.Амар” PER объектыг 0.94 магадлалын оноотойгоор таньдаг болов.

Preprocessing аргуудын давуу ба сул тал

Арга	Давуу тал	Сул тал
Нэг горимт preprocessing	Энгийн, хялбар	NOT болон TM аль нэгд тохиромжгүй
Давхар горимт	Даалгавар бүрт оновчтой	Нэг текстийг хоёр удаа боловсруулна (2× хугацаа)
Нэр хамгаалах	Монгол нэрийн таних чадвар +30%	Regex-ийн нарийвчлал шаардана

Хүснэгт 3.4: Preprocessing аргуудын харьцуулалт

3.4 Нэрлэсэн объект таних загварын хэрэгжүүлэлт ба нарийвчилсан сургалт

Шаардлагатай үед ачаалах ба багцлан боловсруулах архитектур

HuggingFace-ийн BERT загвар (700MB) эхний дуудлагад л ачаалагддаг “шаардлагатай үед ачаалах” аргыг хэрэглэв. Багцлан боловсруулах нь олон текстийг нэг дор GPU/CPU-д дамжуулж боловсруулдаг тул нэг-нэгээр дамжуулахаас 4 дахин хурдан байна.

```
HF_MODEL_ID = "Nomio4640/ner-mongolian"
MAX_CHUNK_CHARS = 1_300  # ~400 Монголтокен  < BERT 512 хязгаар

class NEREngine:
    def __init__(self, model_name: str = None):
        local_path = os.path.join(
            os.path.dirname(os.path.dirname(__file__)),
            "adapters", "ner_mongolian")
```

```
        if model_name:
            self.model_name = model_name           # гараартодорхойлсон
        elif os.path.exists(
                os.path.join(local_path, "model.safetensors")):
            self.model_name = local_path            # локалзагвар
        else:
            self.model_name = HF_MODEL_ID           # HuggingFace Hub
        self._pipeline = None

    def _load_pipeline(self):
        if self._pipeline is None:
            import torch
            from transformers import pipeline
            device = 0 if torch.cuda.is_available() else -1
            self._pipeline = pipeline(
                "ner", model=self.model_name,
                aggregation_strategy="simple",
                device=device,  # GPU боломжтойболавтомат
            )
        return self._pipeline

    def recognize_batch(self, texts: List[str],
                        batch_size: int = 16) -> List[List[
                           EntityResult]]:
        out = [[] for _ in texts]
        short_texts, short_idx, long_idx = [], [], []
        for i, text in enumerate(texts):
            if not text or not text.strip():
                continue
            if len(text) > MAX_CHUNK_CHARS:
                long_idx.append(i)      # урт: chunk-and-merge
            else:
                short_texts.append(text)
                short_idx.append(i)     # богино: GPU batch
        if short_texts:
            pipe = self._load_pipeline()
            raw_results = pipe(short_texts, batch_size=batch_size)
            for idx, raw in zip(short_idx, raw_results):
                out[idx] = [EntityResult(
                    word=e["word"], entity_group=e["entity_group"],
                    score=float(e["score"]),
                    start=int(e["start"]), end=int(e["end"]),
```

```
                ) for e in raw]
        for idx in long_idx:
            out[idx] = self._recognize_chunked(texts[idx])
        return out
```

Listing 3.4: NEREngine — шаардлагатай үед ачаалах ба багцлан боловсруулах

HOT туршилтын үр дүн (нарийвчлан сургах хийхээс өмнө)

Суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`)-ыг Монгол мэдээний 500 нийтлэл дээр объектын түвшний яг таарсан аргаар үнэлэхэд дараах үр дүн гарав:

Үзүүлэлт	Precision	Recall	F1	Тайлбар
Суурь загвар (500 нийтлэл)	0.321	0.316	**0.318**	нарийн тааруулга хийгдээгүй

Хүснэгт 3.5: HOT суурь загварын гүйцэтгэлийн үнэлгээ (eval/evaluate_tokens.py, 500 нийтлэл)

- **True Positives:** 226 — зөв олсон объект
- **False Positives:** 478 — буруу таньсан объект
- **False Negatives:** 489 — алдсан объект

F1=0.318 нь бодит хэрэглээнд тохирохгүй бага үзүүлэлт юм. Гол шалтгаанууд нь: (1) загвар Монгол хэл дээр тусгайлан нарийвчлан сургах хийгдээгүй, (2) Монгол нэрийн “А.Бат-Эрдэнэ” хэлбэрийг хуваан таньдаг, (3) “МАН”, “УИХ” зэрэг Монгол товчлолыг мэддэггүй.

Давуу тал: Монгол хэлний залгаварт бүтцийг BERT subword tokenization зохицуулж чаддаг; багцлан боловсруулах нь GPU дээр 20–60 ms латентитэй. **Сул тал:** нарийн тааруулга хийхгүйгээр Монгол текст дээр F1=0.318 — домайн-специфик нэршил, товчлол алдагддаг.

Суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`) нь Монгол хэл дээр тусгайлан сургаагүй тул хэд хэдэн чухал дутагдалтай байв. Тухайлбал “Л.Оюун-Эрдэнэ” нэрийг “Л” болон “Оюун-Эрдэнэ” гэж хоёр тусдаа объект болгон хуваадаг, “МАН” зэрэг Монгол байгууллагын товчлолыг таньдаггүй, мөн газрын нэрийн нарийн ялгааг сайн ялгадаггүй байсан. Эдгээр асуудлыг шийдэхийн тулд хоёр үе шаттай нарийвчилсан сургалт хийв.

Асуудлын тодорхойлолт — Tokenization ба тэмдэглэлийн зөрчил

BERT загварын WordPiece токенжуулагч нь текстийг дэд-токен болгон хуваадаг. Монгол хэлний нэрийн “Д.Батулга” хэлбэрт энэ нь онцгой асуудал үүсгэдэг:

```
Оригинал
: ДБатулга   "."Токенууд
: Д   ["", ".", Батулга""]     (3 токен)

V1 тэмдэглэл:Д→
           B−PER
  . →        −100 ←загвар     "." тэмдэгтийгнэрийндоторБатулга→
     −100 гэдгийгхэзээчсуралцдаггүй         !

V2 тэмдэглэлтэмдэгтбайрлалын (−):Д→
           B−PER
  . →        I−PER ←загвар    "." ньнэрийндоторБатулга→
     I−PER байжболохыгсуралцдаг       !
```

Listing 3.5: WordPiece tokenization — цэг дээр хуваагдах жишээ

V1 сургалтын өгөгдөлд зөвхөн эхний дэд-токен тэмдэглэл авч, үлдсэн нь -100 (“үл тооцох”) утга авдаг байсан. Үүний улмаас загвар “`.`” тэмдэгтийг нэрийн дотор байж болохыг хэзээ ч суралцаагүй бөгөөд inference хийхэд “`.`” дээр объектыг таслан хуваадаг байв. Ижил асуудал “`-`” (зураас) тэмдэгт дээр ч тохиолддог байсан (жишээ нь “Оюун-Эрдэнэ”).

V1 сургалт — Суурь нарийвчилсан сургалт

Эхний нарийвчилсан сургалт нь CoNLL форматын (үг-level тэмдэглэл) өгөгдөл ашигласан бөгөөд `Davlan/bert-base-multilingual-cased-ner-hrl` суурь загвараас эхлэн сургасан. Google Colab-ийн T4 GPU дээр 4 epoch сургасан:

Epoch	Training Loss	Val Loss	F1	Precision	Recall
1	0.0875	0.1042	0.773	0.752	0.794
2	0.0587	0.1027	0.801	0.773	0.830
3	0.0390	0.1007	0.811	0.771	0.856
4	0.0259	0.1108	0.809	0.786	0.834

Хүснэгт 3.6: V1 нарийвчилсан сургалт сургалтын үр дүн (CoNLL word-level тэмдэглэл)

V1 загвар F1=0.811 хүрсэн нь суурь загвараас мэдэгдэхүйц сайжирсан боловч Монгол нэрийн “А.Бат-Эрдэнэ” хэлбэрийг зөв таньдаггүй сул талтай байсан. Eval скрипт (`eval/evaluate_tokens.py`)-ийг Монгол мэдээний 500 нийтлэл дээр ажиллуулахад объектын түвшний F1=0.32 гарсан нь бодит хэрэглээнд тохирохгүй байгааг баталсан.

V2 сургалтын өгөгдлийн бэлтгэл

V2 сургалтын өгөгдлийг бэлтгэхэд дараах алхмуудыг хэрэгжүүлэв:

Тэмдэглэлийн формат — Тэмдэгт-байрлалын (Character-offset)

Word-level тэмдэглэлийн оронд тэмдэгт-байрлалын (тэмдэгт-хэсгийн) тэмдэглэлийг ашиглав. Энэ форматад объектын эхлэл ба төгсгөлийн тэмдэгтийн байрлалыг (start, end) тодорхойлж, тухайн хэсэгт багтах **бүх** дэд-токенд BIO тэмдэглэл өгдөг:

```
{
  "text": Ерөнхий" сайдЛОюунЭрдэнэ .– мэдэгдэлхийлээ ",
  "labels": [[13, 27, "PER"]]
}
// Тэмдэгтбайрлал  [13:27] = ЛОюунЭрдэнэ".–"
// Бүхдэдтокен – Л(, ., Оюун, –, Эрдэнэ) B–PER/I–PER авна
```

Listing 3.6: Тэмдэгт-байрлалын (Character-offset) JSON формат

Өгөгдлийн эх сурвалжууд

Сургалтын өгөгдлийг олон эх сурвалжаас нэгтгэн үүсгэв:

Эх сурвалж	Тоо	Тайлбар
NER_v1.0.json.gz	10,162	Монгол НОТ корпус — гар тэмдэглэлт, PER/ORG/LOC/MISC
Монгол нэрс	120+	Түгээмэл эрэгтэй, эмэгтэй нэрс
Улс төрчид	30+	Одоогийн бодит нэр, үсгийн товчлолтой (Л.Оюун-Эрдэнэ, У.Хүрэлсүх г.м.)
Товчлолууд	526	Монгол байгууллагын товчлол ба бүтэн нэр (УИХ, МАН г.м.)
Засгийн газрын байгууллага	30+	Яам, агентлагийн бүтэн нэр
Компани / банк / хэвлэл мэдээлэл	50+	Голомт банк, Мобиком, Өрнөдөс ТВ г.м.
Газар нэр (level 2–5)	530+	22 аймаг, 26 сум, 89 хороо, 142 хороолол

Хүснэгт 3.7: V2 сургалтын өгөгдлийн эх сурвалжууд

Өгөгдлийн үүсгэлтийн онцлог шийдлүүд

1. **Цэгтэй нэрийн загвар:** “Д.Батулга”, “Л.Оюун-Эрдэнэ” зэрэг Монгол нэрийн стандарт хэлбэрийг олон тооны template өгүүлбэрт оруулж, загварыг цэгийн дотор объект үргэлжилдгийг суралцуулав.

2. **Залгаварт нэрийн хэлбэр:** Монгол хэлний “-ын”, “-ийн”, “-д”, “-г” гэсэн нөхцөл залгасан нэрүүдийг (“Батулгыг”, “Хүрэлсүхийн”) тусгайлан оруулав.

3. **Товчлол + бүтэн нэр хослол:** “МАН” товчлол болон “Монгол Ардын Нам” бүтэн нэрийг аль алиныг нь ORG объектад сургав.

4. **Хольсон объект өгүүлбэрүүд:** Нэг өгүүлбэрт PER + ORG + LOC гурвуулаа орсон жишээнүүд оруулж, загварыг олон объекттэй контекстэд ажиллахад сургав.

Нэгтгэсэн өгөгдлийн статистик

Үзүүлэлт	**Утга**
Нийт өгүүлбэр	20,696
LOC тэмдэглэл	11,246
ORG тэмдэглэл	10,933
PER тэмдэглэл	9,603
MISC тэмдэглэл (O болгон хөрвүүлсэн)	2,839
Цэгтэй нэр объект	6,862
Зураастай нэр объект	1,185
Нөхцөл залгасан нэр объект	8,308
Валидацийн алдаа	0

Хүснэгт 3.8: V2 сургалтын өгөгдлийн статистик (train_v2_merged.jsonl)

V2 сургалтын тохиргоо ба явц

V2 нарийвчилсан сургалт нь V1 загвараас үргэлжлүүлэн сургасан (continued нарийвчилсан сургалт). Суурь загвараас шинээр эхлэхийн оронд V1-д суралцсан мэдлэг дээр нэмж сургав.

Сургалтын тохиргоо

Тэмдэгт-байрлалын (Character-offset) tokenization алгоритм

V2 сургалтын гол шинэлэг шийдэл нь тэмдэгт-байрлалын-д суурилсан токенчлолын тааруулга юм. Уг алгоритм нь HuggingFace токенжуулагч-ийн `return_offsets_mapping`

Параметр	V1	V2	Тайлбар
Суурь загвар	Davlan BERT	Nomio4640/ ner-mongolian (V1)	V2 нь V1-ээс үргэлжлүүлсэн
Learning rate	3e-5	2e-5	Үргэлжлүүлэн сургахад бага LR
Epoch	4	5	Шинэ өгөгдлийн хэмжээнд тохируулсан
Batch size	16	16	T4 GPU-ийн санах ойд тохирсон
Warmup steps	200	300	Илүү их өгөгдөлтэй тул нэмсэн
Weight decay	0.01	0.01	Ижил регуляризаци
FP16	Тийм	Тийм	Хагас нарийвчлалтай сургалт
Тэмдэглэлийн формат	CoNLL word-level	JSON тэмдэгт-байрлалын	Бүх дэд-токен тэмдэглэл авна
Сургалтын өгүүлбэр	∼8,000	20,696	2.5 дахин их өгөгдөл
Валидацийн өгүүлбэр	915	915	Ижил eval set

Хүснэгт 3.9: V1 ба V2 сургалтын тохиргооны харьцуулалт

функцийг ашиглан токен бүрийн анхны текст дэх тэмдэгтийн байрлалыг авч, объект span-д харьцуулдаг:

```
def tokenize_json_entry(entry):
    text = entry['text']
    label_spans = entry['labels']  # [[start, end, label], ...]
    encoding = токенжуулагч(text, truncation=True, max_length=512,
                        return_offsets_mapping=True)
    offset_mapping = encoding['offset_mapping']
    token_labels = []
    for idx, (tok_start, tok_end) in enumerate(offset_mapping):
        if tok_start == 0 and tok_end == 0:
            token_labels.append(-100)  # [CLS], [SEP] тусгайтокен
            continue
        label = "O"
        for span_start, span_end, span_label in label_spans:
            if tok_start >= span_start and tok_end <= span_end:
                # Өмнөхтокенмөн  entity доторбол  I-, үгүйбол  B-
                is_first = True
                if idx > 0:
                    prev_s, prev_e = offset_mapping[idx - 1]
```

```
                    if prev_s >= span_start and prev_e <= span_end:
                        is_first = False
                label = f"B-{span_label}" if is_first else f"I-{
                    span_label}"
                break
        token_labels.append(label2id.get(label, 0))
    encoding['labels'] = token_labels
    return encoding
```

Listing 3.7: Тэмдэгт-байрлалын (Character-offset) tokenization тааруулга

Энэ алгоритмын гол давуу тал нь объект span дотор орших **бүх** тэмдэгт — “.”, “-”, дэд-үг (subword) зэрэг — B-/I- тэмдэглэл авдаг. Word-level аргаас ялгаатай нь -100 (үл тооцох) утга зөвхөн тусгай токен ([CLS], [SEP], [PAD]) болон объектын гаднах padding-д хэрэглэгддэг.

V2 сургалтын үр дүн

Google Colab T4 GPU дээр 5 epoch, нийт 46 минутын сургалт хийв:

Epoch	Training Loss	Val Loss	F1	Precision	Recall
1	0.0388	0.2818	0.797	0.732	0.876
2	0.0195	0.3036	0.822	0.756	0.901
3	0.0147	0.3283	0.833	0.763	0.917
4	0.0083	0.3422	0.843	0.776	0.924
5	**0.0048**	**0.3629**	**0.845**	**0.778**	**0.925**

Хүснэгт 3.10: V2 нарийвчилсан сургалт сургалтын үр дүн (тэмдэгт-байрлалын тэмдэглэл)

Сургалтын явцын шинжилгээ

Training Loss — Суралцалтын явц

Training loss нь epoch бүрт тасралтгүй буурч (0.0388 → 0.0048) загвар сургалтын өгөгдлийг маш сайн суралцаж байгааг харуулав. Epoch 5-д loss 0.0048 хүрсэн нь загвар сургалтын өгүүлбэрүүдийн объектыг бараг бүрэн зөв таньж байгааг илтгэнэ.

Validation Loss — Ерөнхийлөх чадвар

Validation loss нь epoch 1-ийн 0.2818-аас epoch 5-д 0.3629 болж нэмэгдсэн. Энэ нь дараах шалтгаантай:

1. **Тархалтын зөрүү:** Сургалтын өгөгдөл нь үүсгэсэн өгүүлбэрүүд, харин валидацийн өгөгдөл нь бодит текстээс авсан CoNLL формат. Хоёр өгөгдлийн хэв маяг ялгаатай.

2. **Тэмдэглэлийн формат зөрүү:** Сургалтын өгөгдөлд “.” нь I-PER тэмдэглэл авдаг, харин валидацийн өгөгдөлд −100 авдаг. Загвар шинэ зүйл суралцаж байгаа ч хуучин eval set үүнийг бүрэн хэмжээгүй.

3. **Хэмжээний зөрүү:** Сургалтын 20,696 ба валидацийн 915 өгүүлбэрийн харьцаа (96:4) нь стандарт 80:20 харьцаанаас зөрүүтэй.

Гэвч F1 оноо тасралтгүй өссөн нь validation loss-оос илүү чухал үзүүлэлт юм — загвар зөв хариулт гаргаж байгаа ч итгэлцлийн тархалт хуучин eval set-тэй бүрэн нийцэхгүй байна гэсэн үг.

Precision ба Recall-ийн тэнцвэр

- **Recall** (0.925) нь **Precision** (0.778)-ээс мэдэгдэхүйц өндөр. Энэ нь загвар бодит объектын 92.5%-ийг олдог боловч олсон объектын 22.2% нь буруу байна гэсэн үг.
- Сошиал мониторингийн хэрэглээнд recall өндөр байх нь чухал — объект орхигдуулахаас илүү нэмэлт объектыг дараа нь шүүх нь хялбар.

V1 ба V2 загварын харьцуулалт

Үзүүлэлт	Суурь загвар	V1	V2
F1 Score	0.318	0.811	**0.845**
Precision	0.321	0.786	0.778
Recall	0.316	0.834	**0.925**
“Д.Батулга” таних	2 объект	2 объект	**1 объект**
“Л.Оюун-Эрдэнэ” таних	3 объект	2–3 объект	**1 объект**
“МАН” ORG таних	Үгүй	Тийм	**Тийм**
Нөхцөл залгасан нэр	Муу	Дунд	**Сайн**

Хүснэгт 3.11: Суурь загвар, V1, V2 гүйцэтгэлийн харьцуулалт
* Суурь загварын F1=0.318 нь Монгол мэдээний 500 нийтлэлийн объектын түвшний яг таарсан үнэлгээ

Туршилтын жишээ: Суурь загвар ба V2 загварын харьцуулалт

Ижил 5 өгүүлбэр дээр суурь загвар (`Davlan/bert-base-multilingual-cased-ner-hrl`) ба нарийвчлан сургах хийсэн V2 загвар (`Nomio4640/ner-mongolian`)-ыг туршив:

Хүснэгтээс харахад суурь загвар нь Монгол нэрийн бүтцийг огт ойлгодоггүй — “Л.Оюун-Эрдэнэ” нэрийг цэгээр нь хувааж, “Эрдэнэ”-г LOC гэж буруу ангилдаг; “МАН” товчлолыг “М” (LOC) ба “##АН” (ORG) гэж задалдаг; “Батулгыг” нөхцөлтэй нэрийг огт

Оролтын текст	**Суурь загвар**	**V2 загвар**
Л.Оюун-Эрдэнэ ерөнхий сайд	Л. Оюун → PER (0.999) Эрдэнэ → LOC (0.990) *2 объект, буруу LOC*	**Л. Оюун-Эрдэнэ → PER (1.000)** *1 объект, зөв*
Пурэвдоржийн Номин-Эрдэнэ МУИС-ын оюутан	Пурэвдор → PER (0.822) ##мин → PER (0.862) Э → LOC (0.558) ##рдэнэ → PER (0.697) МУИС-ын → ORG (0.849) *5 хэсэг болж задарсан*	**Пурэвдоржийн Номин-Эрдэнэ → PER (1.000)** **МУИС → ORG (1.000)** *2 объект, бүгд зөв*
Батулгыг шүүхэд дуудсан	*(объект олдсонгүй)*	**Батулгыг → PER (1.000)**
МАН-ын дарга	М → LOC (0.500) ##АН → ORG (0.558) *хуваагдсан, буруу LOC*	**МАН → ORG (1.000)**
УИХ-ын гишүүн Д.Ганбат	УИХ-ын → ORG (0.947) Д. Ганбат → PER (1.000)	**УИХ → ORG (0.829)** **Д. Ганбат → PER (1.000)**

Хүснэгт 3.12: Суурь загвар ба V2 загварын бодит туршилтын харьцуулалт (5 жишээ)

таньдаггүй. V2 загвар нь эдгээр бүх тохиолдлыг зөв, өндөр итгэлтэйгээр (0.829–1.000) таньж байна.

Загварын байршуулалт ба синхрончлол

нарийн тааруулга хийсэн загварыг HuggingFace Hub (`Nomio4640/ner-mongolian`) руу байршуулж, гурван орчинд автоматаар ачаалагдах архитектур хэрэгжүүлэв:

```
HF_MODEL_ID = "Nomio4640/ner-mongolian"

class NEREngine:
    def __init__(self, model_name: str = None):
        local_path = os.path.join(os.path.dirname(__file__),
                                  "..", "adapters", "ner_mongolian")
        if model_name:
            self.model_name = model_name          #
               гараартодорхойлсон
        elif os.path.exists(
                os.path.join(local_path, "model.safetensors")):
            self.model_name = local_path           # локалзагвар
        else:
            self.model_name = HF_MODEL_ID          # HuggingFace Hub
```

Listing 3.8: NEREngine — автомат загвар илрүүлэх механизм

Энэ механизмаар:

- **Локал хөгжүүлэлт:** `adapters/ner_mongolian/` дотор загвар байвал локалаас ачаална
- **Google Colab + ngrok:** Репо clone хийгдэхэд загвар байхгүй тул HuggingFace Hub-аас татна
- **HuggingFace Space:** Мөн HuggingFace Hub-аас автоматаар татна

Загварын жинг GitHub-д хадгалахгүй (.gitignore-д нэмсэн) бөгөөд зөвхөн HuggingFace Hub-д хадгалдаг. Ингэснээр GitHub-ийн 100MB файлын хязгаарлалтаас зайлсхийж, кодын репо болон загварын жинг тусад нь удирдах боломжтой болсон.

Нарийвчилсан сургалт-ийн давуу ба сул тал

Давуу тал	Сул тал
F1 оноо 0.32 → 0.845 (163% сайжралт)	Валидацийн өгөгдлийн хэмжээ бага (915)
Цэгтэй нэрийг бүтнээр таньдаг болсон	Үүсгэсэн өгөгдөл бодит текстээс ялгаатай
Recall 0.925 — объект бараг алдагддаггүй	Precision 0.778 — false positive бий
HuggingFace Hub-аар хялбар байршуулдаг	Сургалт GPU шаарддаг (Colab T4)
Олон орчинд автомат синхрончлогддог	Загварын хэмжээ 700MB — татах хугацаа шаардагдана

Хүснэгт 3.13: НОТ нарийвчилсан сургалт-ийн давуу ба сул тал

3.5 Сэдэв загварчлалын хэрэгжүүлэлт

Сэдэв загварчлалын аргуудын харьцуулалт

Сэдэв загварчлалын гурван үндсэн аргыг Монгол хэлний текстэд хэрэглэх боломжийг харьцуулан судалсан. Хүснэгт 3.14-д LDA, BERTopic (HDBSCAN), болон BERTopic (KMeans) аргуудыг гол шалгууруудаар харьцуулав.

Аргын сонголтын үндэслэл: LDA нь Bag-of-Words-д суурилдаг тул Монгол хэлний залгаварт бүтцэд (нэг үгийн олон хэлбэр: “монголын”, “монголд”, “монголоос”) сул ажиллана. Мөн сэдвийн тоо K-г урьдчилан тодорхойлох шаардлагатай нь бодит хэрэглээнд тохиромжгүй. BERTopic + HDBSCAN нь семантик вектор дүрслэлд суурилан автоматаар сэдэв илрүүлдэг давуу талтай боловч 50-аас цөөн баримт бичгийн хувьд бүх текстийг гажиг (хэвийн бус) утга буюу outlier (-1) болгодог ноцтой дутагдалтай. Иймд манай системд **BERTopic + KMeans**-ийг сонгож, `min_topics` ба `max_topics` параметрээр сэдвийн тоог хянах боломжтой болгосон.

Хүснэгт 3.14: Сэдэв загварчлалын аргуудын харьцуулалт

Шалгуур	**LDA**	**BERTopic + HDBSCAN**	**BERTopic + KMeans**
Суурь арга	Bag-of-Words, Dirichlet тархалт	Transformer вектор дүрслэл + нягтрал кластерчлал	Transformer вектор дүрслэл + K тооны кластер
Сэдвийн тоо (K)	Урьдчилан тодорхойлох шаардлагатай	Автоматаар тодорхойлогдоно	`min_topics`, `max_topics` параметрээр тохируулна
Семантик ойлголт	Сул — зөвхөн үгийн давтамж	Хүчтэй — утгын ойролцоо байдлаар бүлэглэнэ	Хүчтэй — утгын ойролцоо байдлаар бүлэглэнэ
Жижиг датасет (<50 док)	Тогтвортой ажилладаг	Бүтэлгүй — бүх баримт бичиг гажиг (хэвийн бус) утга буюу outlier (-1) болдог	Сайн ажиллана, гажиг (хэвийн бус) утга буюу outlier үүсгэхгүй
Том датасет (>500 док)	Сайн	Маш сайн — гажиг (хэвийн бус) утга буюу outlier шүүх чадвартай	Сайн — гэхдээ гажиг (хэвийн бус) утга буюу outlier илрүүлэхгүй
Монгол хэлний дэмжлэг	Шаардлана: стемминг, хасагдах үг жагсаалт	Multilingual вектор дүрслэл автоматаар зохицуулна	Multilingual вектор дүрслэл + custom токенжуулагч
Тооцооллын хурд	Хурдан (CPU: <1 сек / 500 док)	Удаан (вектор дүрслэл: 30–60 сек / 500 док)	Дунд (вектор дүрслэл: 30–60 сек, кластер: <1 сек)
Сонголтын үндэслэл	**BERTopic + KMeans-ийг сонгосон** — сэдвийн тоог хянах, семантик ойлголт, жижиг/том датасет аль алинд тогтвортой		

BERTopic pipeline-ийн бүтэц

BERTopic нь дараах дөрвөн үе шатаас бүрддэг pipeline-аар ажилладаг бөгөөд бид үе шат бүрт Монгол хэлний тусгай тохиргоо хийсэн:

1. **Вектор дүрслэл үүсгэх:** `paraphrase-multilingual-mpnet-base-v2` загвар ашиглан текст бүрийг 768 хэмжээст вектор руу хөрвүүлнэ. Энэ загвар нь 50+ хэл дэмждэг тул Монгол текстийг тусад нь нарийвчлан сургах хийх шаардлагагүй.

2. **UMAP хэмжээс бууруулалт:** 768 хэмжээст вектор дүрслэлийг 5 хэмжээст руу бууруулж, бүлэглэлтийн өмнө чимээг арилгана.

3. **Бүлэглэлт (KMeans):** Бууруулсан вектор дүрслэл дээр KMeans алгоритм

ажиллуулж, текстүүдийг сэдвийн бүлэгт хуваарилна. Сэдвийн тоог дараах томьёогоор тооцно (default: 5–15):

$$n = \max(\texttt{min_topics},\ \min(N/10,\ \texttt{max_topics}))$$

4. **c-TF-IDF сэдвийн дүрслэл:** Кластер бүрийн хамгийн онцлог үгсийг c-TF-IDF ашиглан олж, MaximalMarginalRelevance (MMR)-ээр олон янзтай keyword сонгоно.

Монгол хэлний тусгай тохиргоо

BERTopic-ийн c-TF-IDF алгоритм нь CountVectorizer ашиглан үг тоолдог тул Монгол хэлний залгаварт дагавар нь нэг ойлголтыг олон токен болгон хуваадаг (жишээ нь: “монголын”, “монголд”, “монголоос” → гурван тусдаа токен). Үүнийг шийдэхийн тулд:

- **Suffix stripping:** 13 төрлийн Монгол дагаварыг (харьяалах: -ын/-ийн, гарах: -аас/-ээс, чиглэх: -руу/-рүү, олон тоо: -ууд/-үүд г.м.) таслан root хэлбэрт шилжүүлнэ. Root нь ≥ 3 тэмдэгт байх ёстой.
- **Хасагдах үг шүүлтүүр:** 120+ Монгол хасагдах үг (“байна”, “болох”, “гэж”, “энэ” г.м.) болон мэдээний нийтлэг үгс (“аймаг”, “хувь”, “тэрбум”)-ийг шүүнэ.
- **Custom токенжуулагч:** Дээрх хоёр алхмыг нэгтгэсэн `_mongolian_токенжуулагч` функцыг CountVectorizer-т дамжуулна.

```
_MN_SUFFIXES_MULTI = [
    "аас", "ээс", "оос", "өөс",           # Гарах
    "дийн", "ийн", "дын", "ний", "ын",    # Харьяалах
    "аар", "ээр", "оор", "өөр",            # Зэвсгийн
    "тай", "тэй", "той",                   # Хамтрах
    "руу", "рүү",                          # Чиглэх
    "нууд", "нүүд", "ууд", "үүд",         # Олонтоо
    "ийг", "ыг",                           # Заах
]
_MN_SUFFIXES_SHORT = ["д", "т"]  # зөвхөнэгшигдуулт /
   гийгүүлэгчийндараа
_CYRILLIC = set("абвгдеёжзийклмнопрстуфхцчшщъыьэюяөү"
                "АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯӨҮ")

def _mongolian_tokenizer(text: str) -> List[str]:
    tokens = []
    for w in text.split():
        if not w or len(w) < 2 or w.isdigit():
            continue
        if not any(c in _CYRILLIC for c in w):
```

```
            continue            # Латинангли/ токеныгхасна
        stem = _mn_stem(w)
        if stem.lower() in _MN_STOPWORDS or len(stem) < 3:
            continue
        tokens.append(stem)
    return tokens

vectorizer = CountVectorizer(
    tokenizer=_mongolian_tokenizer,
    min_df=1, max_df=0.80,
)
```

Listing 3.9: Монгол хэлний custom токенжуулагч — suffix stripping + хасагдах үг

Адаптив кластерчлал — KMeans fallback механизм

BERTopic-ийн HDBSCAN кластерчлал нь default тохиргоонд `min_cluster_size=10` байдаг тул 50-аас цөөн баримт бичгийн бүх текст -1 (гажиг (хэвийн бус) утга буюу outlier) болж буцдаг. Энэ асуудлыг шийдэхийн тулд адаптив кластерчлал хэрэгжүүлсэн:

```
MIN_BERTOPIC_DOCS = 50    # аас50- доош: KMeans (HDBSCAN хангалтгүй)
MIN_TINY_DOCS     = 3     # аас3- доош: загварчлалогтхийхгүй

class TopicModeler:
    def __init__(self, min_topics=5, max_topics=15):
        self.min_topics = min_topics
        self.max_topics = max_topics

    def _make_bertopic(self, n_docs: int, embeddings=None):
        if n_docs >= MIN_BERTOPIC_DOCS:
            # HDBSCAN: томдатасетэд  outlier автоматааршүүнэ
            cluster_model = HDBSCAN(
                min_cluster_size=5,
                cluster_selection_method="leaf")
        else:
            # KMeans: жижигдатасетэд  outlier үүсгэхгүй
            k_min = max(2, self.min_topics)
            k_max = min(self.max_topics, max(2, n_docs // 2))
            if embeddings is not None:
                # Silhouette оноогооороновчтой  K хайна
                n_clusters = self._find_optimal_k(
                    embeddings, range(k_min, k_max + 1))
            else:
```

```
                n_clusters = max(k_min, min(k_max, n_docs // 3))
            n_clusters = min(n_clusters, n_docs)
            cluster_model = KMeans(n_clusters=n_clusters,
                                   random_state=42, n_init="auto")
        return BERTopic(
            embedding_model=self._load_embedding_model(),
            hdbscan_model=cluster_model,
            vectorizer_model=vectorizer,   # Монголтокенжуулагч
            representation_model=MMR(diversity=0.5),
        )
```

Listing 3.10: Адаптив кластерчлал — KMeans + тохируулж болох сэдвийн тоо

Хүснэгт 3.15: Датасетийн хэмжээнээс хамаарсан сэдвийн тооны тооцоо

Баримт бичгийн тоо	**Сэдвийн тоо**	**Тооцоолол**
10	5	$\max(5, \min(10/3, 15)) = 5$
50	5	$\max(5, \min(50/10, 15)) = 5$
100	10	$\max(5, \min(100/10, 15)) = 10$
500	15	$\max(5, \min(500/10, 15)) = 15$
1000	15	$\max(5, \min(1000/10, 15)) = 15$ (дээд хязгаар)

Сэдэв загварчлал туршилтын үр дүн

500 Монгол мэдээний нийтлэл дээр хийсэн туршилтад BERTopic + KMeans нь 15 сэдэв илрүүлсэн. Хүснэгт 3.16-д гол сэдвүүдийг тоймлов.

ID	Сэдвийн нэр	Гол түлхүүр үгс	Док тоо
0	Түгжрэл, залуучууд, эрүүл мэнд	түгжрэл, залуучууд, мэнд, хүртээмж	76
1	Блокчэйн, стартап, компани	блокчэйн, стартап, компани, боломж	73
2	Комисс, эрх, тайлан, алт	комисс, эрх, тайлан, алт	41
3	Өсөх, сүү, үйлдвэр, орлого	өсөх, сүү, үйлдвэр, орлого	40
4	Үнэ, тогтолцоо, олгогдож, нэвтэрч	үнэ, тогтолцоо, олгогдож	40
5	Инноваци, үнэ, тэжээл, хөдөө	инноваци, үнэ, тэжээл, хөдөө	38
6	Алтны олборлол, үргэлжлэл	алтны, олборлол, үргэлжилж, амжилт	31
		... нийт 15 сэдэв	**500**

Хүснэгт 3.16: BERTopic + KMeans-ийн илрүүлсэн гол сэдвүүд (500 нийтлэл)

Үр дүнгийн шинжилгээ

- **Семантик ялгарал:** Вектор дүрслэлд суурилсан кластерчлал нь “блокчэйн стартап” ба “IT компани” сэдвүүдийг ялгаж чадсан — LDA-д ийм ялгаа гарахгүй, учир нь хоёулаа “компани” гэдэг нийтлэг үг агуулдаг.
- **Custom токенжуулагч-ийн нөлөө:** Suffix stripping-гүй тохиолдолд “малчдын”, “малчид”, “малчдад” гурван тусдаа keyword болдог байсан бол одоо “малч” нэг keyword болж нэгдсэн.
- **Хасагдах үгийн нөлөө:** “байна”, “болох”, “аймаг” зэрэг үгсийг шүүснээр сэдвийн keyword-ийн мэдээллийн агуулга нэмэгдсэн.
- **Хурд:** GPU (T4) дээр 500 текстийн вектор дүрслэл 8–12 секунд, KMeans кластерчлал <1 секунд, нийт pipeline 10–15 секунд зарцуулсан.

Давуу тал: Урьдчилан мэдэгдэхгүй шинэ сэдэв автоматаар илрүүлдэг; вектор дүрслэлд суурилсан тул ижил утгатай олон хэлбэрийн үгсийг нэг кластерт хамруулна; `min_topics`/`max_topics` параметрээр сэдвийн тоог хянах боломжтой. **Сул тал:** Вектор дүрслэл тооцоолол нь CPU дээр 30–60 секунд зарцуулдаг (GPU шаардлагатай); KMeans нь гажиг (хэвийн бус) утга буюу outlier илрүүлэхгүй тул шуугиантай баримт бичгийг ч ямар нэг сэдэвт оноодог.

Сэдэв загварчлал тохиргооны харьцуулалт — Coherence ба Topic Diversity

Системийн оновчтой тохиргоог тогтоохын тулд хоёр тохиргооны BERTopic pipeline-ийг 200 Монгол мэдээний нийтлэл дээр туршин, Coherence (CV), NPMI болон Topic Diversity

хэмжүүрүүдээр харьцуулан үнэллээ.

Тохиргоо	Coherence (CV)	NPMI	Topic Diversity	Сэдвийн тоо
1. Суурь тохиргоо (paraphrase-mpnet)	**0.549**	-0.063	**0.925**	8
2. Сайжруулсан тохиргоо (Auto-UMAP + Silhouette)	0.473	-0.073	0.912	8

Хүснэгт 3.17: BERTopic тохиргооны харьцуулалт — Coherence ба Diversity хэмжүүрүүд (200 нийтлэл)

Хүснэгт 3.17-ийн дүнгээс харахад суурь тохиргоо (paraphrase-multilingual-mpnet-base-v2 вектор дүрслэл + Auto KMeans) нь Coherence CV=0.549, Topic Diversity=0.925 үзүүлэлтүүдэд хамгийн сайн үр дүн харуулсан. Сайжруулсан тохиргооны (Auto-UMAP + Silhouette оновчтойлол) Coherence дорогшилсон нь олон UMAP тохиргоо туршихад нийт вектор дүрслэлийн тархалтад сөргөөр нөлөөлсөнтэй холбоотой. Иймд **суурь тохиргоог** үндсэн pipeline-д ашиглахаар тогтов.

3.6 Системийн архитектур: FastAPI болон REST API хэрэгжүүлэлт

Hexagonal Architecture-ийн дагуу FastAPI нь NLP Domain Core-ийн гаднах “adapter” болж ажилладаг. `main.py`-д дөрвөн router бүртгэгдсэн:

```
app = FastAPI(title="NLP Intelligence API",
              version="1.0.0", lifespan=lifespan)

app.add_middleware(CORSMiddleware,
    allow_origins=["http://localhost:3000",
                   "http://127.0.0.1:3000"],
    allow_methods=["*"],
    allow_headers=["*"],
)

# Routerүүд: бүх NLP болонудирдлагын  endpointууд
app.include_router(analysis.router,
                   prefix="/api",       tags=["Analysis"])
app.include_router(dashboard.router,
                   prefix="/api",       tags=["Dashboard"])
app.include_router(admin.router,
                   prefix="/api/admin", tags=["Admin"])
```

```
app.include_router(crawler.router,
                   prefix="/api",       tags=["Crawler"])
app.include_router(developer.router,
                   prefix="/api/nlp",   tags=["Developer"])
```

Listing 3.11: FastAPI adapter — main.py роутер бүртгэлт

Router-үүдийн тайлбар:

- **analysis.py:** НОТ, Сэдэв загварчлал болон бүрэн шинжилгээний endpoint-ууд
- **dashboard.py:** KPI статистик, timeline, mentions feed, топ объект, топ сэдэв (MindMap) endpoint-ууд
- **admin.py:** Стоп үгс, мэдлэгийн сан, шошго удирдах endpoint-ууд
- **crawler.py:** Мэдээний сайтаас нийтлэл цуглуулах endpoint

API-ийн туршилтын жишээ

`POST /api/analyze` endpoint-д 100 нийтлэл илгээсэн туршилтын хариу:

```
$ curl -X POST http://localhost:8000/api/analyze \
  -H "Content-Type: application/json" \
  -d '{"texts": МАНын["- даргаУлаанбаатартуулзалтхийлээ   .",Монгол
                 " банкбодлогынхүүгнэмлээ   ."]}'

# Хариухэсэг ():
{
  "ner": [
    [{"word":"МАН","entity_group":"ORG","score":0.97},
     {"word":"Улаанбаатар","entity_group":"LOC","score":0.99}],
    [{"word":"Монгол банк","entity_group":"ORG","score":0.95}]
  ],
  "topics": [
    {"topic_id": 2, "name": "засгийн газрынсанхүү ", "score": 0.71},
    {"topic_id": 5, "name": "банк, хүү, зээл", "score": 0.68}
  ],
  "processing_time_ms": 143
}
```

Listing 3.12: curl ашиглан API туршилт

3.7 Хяналтын самбар болон Хэрэглэгчийн интерфейс

Next.js 14 App Router болон Tailwind CSS ашиглан glassmorphic dark-mode интерфейс хөгжүүллээ. Dashboard нь гурван үндсэн хуудастай:

Dashboard хуудас

Мониторингийн үндсэн хуудас нь дараах бүрдэлтэй:

Энэхүү хяналтын самбар нь системийн үндсэн хэрэглээ бөгөөд хэрэглэгчдэд ямар ч техникийн мэдлэг шаардалгүйгээр NLP-ийн нарийн төвөгтэй шинжилгээнүүдийг графикаар харуулдаг. Тайлбарлавал:

- **Dashboard хуудас:** Энд нийт хэчнээн мэдээ цугларсныг мөн тэдгээрийн хэдэн хувь нь эерэг эсвэл сөрөг сэтгэгдэлтэй байгааг харуулсан донат чарт байрлана. Түүнчлэн мэдээллийн урсгалыг огноогоор нь харуулсан диаграммтай байх ба хүмүүс, байгууллага, газрын нэрүүдээс хамгийн их дурдагдсан эхний топ объектуудыг (Н.Учрал, У.Хүрэлсүх, УИХ, Монгол гэх мэт) жагсааж баар чарт хэлбэрээр үзүүлнэ.

- **Сэдвийн мод:** BERTopic + KMeans-ийн илрүүлсэн сэдвүүдийн уялдах холбоосыг харуулсан тархмал диаграмм. Жишээлбэл ”орон, татварын, оны” гэсэн түлхүүр үгстэй сэдэв рүү ороход тухайн сэдэвт багтсан мэдээллүүд дэлгэрэнгүй гарч ирдэг интерактив зохиомжтой.

- **Меншн фийд (Mentions Feed / Сүүлийн дурдалтууд):** Мэдээллийн сайтуудаас татагдсан эх сурвалжуудын гарчиг болон зургийг картаар үзүүлж, эх сурвалж руу шилжих болон тухайн мэдээний сэтгэл хөдлөлийн (эерэг, сөрөг, дундаж) төлөвийг өнгөөр ялган шууд харуулдаг онцлогтой.

- **Шүүлтүүрийн мөр:** Хайлтын текст, платформ (Мэдээ/Instagram/Twitter/Facebook), сэтгэл хөдлөл (Positive/Negative/Neutral), огнооны хүрээ сонголттой

- **KPI карт:** Нийт mention тоо, сэтгэл хөдлөлийн тархалт, өмнөх хугацаатай харьцуулсан хувийн өөрчлөлт

- **1-р мөр:** Timeline chart (өдрөөрх mention тоо), эх сурвалжийн задаргаа, Sentiment gauge (donut ring — positive/negative/neutral)

- **2-р мөр:** Гурван НОТ карт — Хүмүүс (PER), Байгууллага (ORG), Газрын нэр (LOC) тус бүрийн топ объектууд

- **3-р мөр:** Topic MindMap — D3.js ашиглан BERTopic-ийн шатлалт сэдвийн модыг интерактив дүрслэл

- **4-р мөр:** Mentions feed — infinite scroll, нийтлэл бүрийн NLP үр дүнг харуулах

Dashboard хуудас нь ачаалагдахад шинжилгээгүй mention байвал автоматаар NLP pipeline ажиллуулж, явцыг progress banner-ээр харуулдаг.

Admin хуудас

Admin хуудас нь зургаан табтай:

1. **Мэдлэгийн сан:** Нэрлэсэн объект нэмэх/устгах (ангилал, объект төрөл)
2. **Шошго:** NOT шошгоны нэрийг өөрчлөх (PER → “Улс төрч”)
3. **Зогсох үгс:** Хасагдах үг нэмэх/устгах (товшиж устгах)
4. **Мэдээний Crawler:** Бүх мэдээний сайтаас crawl хийх, сайтын жагсаалт харах
5. **Хэмжүүрүүд:** Мэдээг Dashboard руу импортлох, ӨС-ийн статистик

Бүх UI шошго Монгол хэлээр бичигдсэн ба dark/light theme-ийн сонголттой.

Developer хуудас

Developer хуудас нь хөгжүүлэгчдэд зориулсан NLP загварын туршилт болон API интеграцийн лавлагааг нэг дор хангана. Хуудас нь дараах хэсгүүдтэй:

1. **Загварын туршилт:** Дурын текст оруулж NOT болон Сэдэв загварчлал-ийг шууд турших, тохиргоо (загварын version, сэдвийн тоо) сонгох боломж.
2. **API код snippet:** Туршилт дууссаны дараа тухайн дуудлагын яг тохиргоог ашиглан cURL, Python, JavaScript гурван хэлбэрийн бэлэн кодыг автоматаар үүсгэж харуулна; нэг товшилтоор clipboard-д хуулах боломжтой.
3. **API Quickstart каталог:** Таван үндсэн endpoint-ийг accordion хэлбэрээр жагсааж, request body жишээ, олон хэлний код snippet болон шууд туршилтын товч (*Шууд турших*) бүхий интерактив лавлагаа хуудас.
4. **BERT vs LLM харьцуулалт:** Локал BERT загвар болон OpenAI-ийн LLM-ийн хурд, зардлын харьцуулсан шинжилгээний хэсэг.

Энэхүү хуудас нь `/docs` хаягт байрлах FastAPI Swagger-ийг нөхөж, туршилтын үр дүнд суурилсан integration code-г шууд авах боломж олгодог.

3.8 Системийн гүйцэтгэлийн үнэлгээ

End-to-end гүйцэтгэлийн хэмжилт

100 текстийн batch дээр хийсэн гүйцэтгэлийн хэмжилт:

Модуль	**CPU (ms)**	**GPU (ms)**
Preprocessing (100 текст)	12	12
HOT batch (100 текст)	4800	380
BERTopic fit+transform	2100	1200
Нийт	**≈6.9 сек**	**≈1.6 сек**

Хүснэгт 3.18: Модуль бүрийн гүйцэтгэлийн хэмжилт (100 текст, batch)

Hexagonal Architecture-ийн үнэлгээ

Hexagonal загвар нь NLP Core-г FastAPI болон Next.js-ийн технологиос бүрэн тусгаарлав. Үр дүнд:

- NLP Core-ийг нэмэлт өөрчлөлтгүйгээр хоёр хувилбарт (Demo Web Application, Public REST API) ашиглах боломжтой болсон.
- Нэг NLP модуль нэмэхэд зөвхөн шинэ Port + Adapter нэмэх болж, одоо байгаа код өөрчлөгддөггүй (Open/Closed зарчим).
- Unit test нь NLP Core-г тусдаа, FastAPI-гүйгээр туршиж болдог болсон.

Сул тал: Жижиг скриптэд hexagonal загвар хэт нарийн гэж үзэгдэж болно; Port/Adapter слоер нэмэгдсэнээр кодын хэмжээ нэмэгдсэн.

Ингэснээр хэрэгжүүлэлтийн бүлэгт Монгол хэлний сошиал өгөгдөлд тохирсон preprocessing, HOT нарийвчилсан сургалт, Сэдэв загварчлал болон мэдээний crawler-ийн хэрэгжүүлэлтийг туршилтаар баталгаажуулав. HOT загварын F1 оноо 0.318-аас 0.845 болж сайжирсан нь нарийвчилсан сургалт болон нэрийн хамгаалах механизмын үр нөлөөг тодорхой харуулж байна. BERTopic+KMeans-ийн суурь тохиргоо CV=0.549, Diversity=0.925 үзүүлэлтээр хамгийн өндөр нарийвчлалтай байсан бол CPU орчинд нийт pipeline 6.9 секундэд ажиллах боломжтой нь бодит хэрэглээний шаардлагыг хангахуйц гэдгийг нотолж байна.

ДҮГНЭЛТ

Энэхүү бакалаврын судалгааны ажлын хүрээнд Монгол хэлний сошиал мониторингийн системд зориулсан нэрлэсэн объект таних болон сэдэв загварчлалд суурилсан байгалийн хэл боловсруулалтын цогц шийдлийг судалж, хэрэгжүүлэн туршилтаар үнэлэв. Монгол хэлний залгаварт морфологи, галиг бичлэгийн олон хувилбар болон тэмдэглэгдсэн корпусын хомс байдал зэрэг гол сорилтуудыг тодорхойлж, тэдгээрийг шийдвэрлэхэд чиглэсэн хэд хэдэн практик шийдлийг боловсруулан хэрэгжүүлэв.

Судалгааны үндсэн үр дүнгийн хувьд дараах амжилтуудыг тэмдэглэж болно. Монгол хэлний сошиал медиа текстэд тохирсон, нэрийн бичиглэлийг хэвээр нь хадгалдаг хувилбар болон сэдэв загварчлалд зориулан үгийн үндэс хүртэл нь урьдчилан боловсруулах хувилбар бүхий хоёр горимт урьдчилсан боловсруулалтын ажлын урсгалыг хэрэгжүүллээ. `Davlan/bert-base-multilingual-cased-ner-hrl` загварыг 20,696 өгүүлбэр бүхий Монгол хэлний өгөгдлийн багц дээр хоёр үе шаттайгаар нарийвчлан сургаснаар нэрлэсэн объект таних F1 оноог 0.845 болтол хүргэж, “Д.Батулга”, “Л.Оюун-Эрдэнэ” зэрэг Монгол нэрийн онцлог хэлбэрийг таних асуудлыг шийдвэрлэв. Сэдэв загварчлалын хувьд BERTopic болон KMeans алгоритмыг хослуулж, өгөгдлийн хэмжээнд тулгуурлан сэдвийн тоог динамикаар тодорхойлох адаптив механизмыг нэвтрүүлснээр 500 нийтлэлийн туршилтаар утга зүйн хувьд уялдаатай 15 сэдэв автоматаар илрүүлэх боломжтой болсон. Эдгээр бүх модулийг Hexagonal архитектурын зарчмаар нэгтгэснээр байгалийн хэл боловсруулалтын домэйн логик нь дэд бүтцийн технологиудаас бүрэн тусгаарлагдаж, нэг суурь кодоос туршилтын вэб програм болон нээлттэй програмчлалын интерфейс хоёуланг нь ажиллуулах боломжтой болов.

Уламжлалт түлхүүр үгэнд суурилсан аргатай харьцуулахад энэхүү шийдэл хэд хэдэн давуу талыг харуулсан. Нэгдүгээрт, “Монгол банк” болон “банк” зэрэг семантикийн хувьд ялгаатай илэрхийллийг контекстийн хүрээнд зөв таньдаг болсон. Хоёрдугаарт, BERTopic нь урьдчилан тодорхойлоогүй шинэ сэдвүүдийг хяналтгүй сургалтын аргаар автоматаар илрүүлж, мэдлэгийн санг байнга гараар шинэчлэх хэрэгцээг эрс бууруулав. Гуравдугаарт, тооцооллын нөөцийн хувьд том хэлний загварын програмчлалын интерфейстэй харьцуулахад нэг текст боловсруулах зардал 15–20 дахин бага байв.

Цаашдын хөгжүүлэлтийн хувьд дараах чиглэлүүд боломжтой юм. Гар тэмдэглэл хийсэн баталгаажуулалтын өгөгдлийн хэмжээг өргөтгөж, “МАН” болон “Монголын Ардын Нам” зэрэг ижил утгатай нэршлийг нэгтгэх объект нэгтгэлийн модуль нэмснээр нэрлэсэн объект таних F1 оноог 0.90-ээс дээш гаргах боломжтой. Мөн “Ulaanbaatar” болон

“Улаанбаатар”-г нэг байршлын нэрт нэгтгэн боловсруулах галигаас кирил рүү хөрвүүлэгч модуль, нэрлэсэн объект болон сэдвүүдийн өөрчлөлтийг цаг хугацааны дарааллаар харуулж трендийг урьдчилан таамаглах функц нэмэх нь мониторингийн системийн аналитик чадавхийг мэдэгдэхүйц нэмэгдүүлнэ. Байгалийн хэл боловсруулалтын ажлын урсгалыг Apache Kafka эсвэл Redis Queue ашиглан бодит цагийн боловсруулалттай холбох нь системийн хэмжээг өргөтгөх нөхцлийг бүрдүүлнэ.

Энэхүү судалгааны ажил нь Монгол хэлний нэрлэсэн объект таних болон сэдэв загварчлалд BERT-д суурилсан байгалийн хэл боловсруулалтын ажлын урсгалыг амжилттай хэрэглэж болохыг туршилтаар нотолж, цаашдын хөгжүүлэлтийн суурь болох нээлттэй, модульчлагдсан архитектурыг бий болгов. Ялангуяа тэмдэглэгдсэн корпус хомс байгаа нөхцөлд олон хэлний урьдчилан сургасан загварыг нарийвчлан сургах арга зүй нь Монгол хэлний байгалийн хэл боловсруулалтын судалгааны практик хэрэгцээнд хариу болж байгаагаараа ач холбогдолтой юм.

Номзүй

[1] Enkhbold Bataa and Josh Wu. Mongolian nlp: Challenges and resources, 2020.

[2] David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3:993–1022, 2003.

[3] Brandwatch. Brandwatch platform: Technical overview, 2023.

[4] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, et al. Unsupervised cross-lingual representation learning at scale. *arXiv preprint arXiv:1911.02116*, 2020.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2019.

[6] Maarten Grootendorst. Bertopic: Neural topic modeling with a class-based tf-idf procedure. *arXiv preprint arXiv:2203.05794*, 2022.

[7] Andreas M Kaplan and Michael Haenlein. Users of the world, unite! the challenges and opportunities of social media. *Business horizons*, 53(1):59–68, 2010.

[8] Meltwater. Meltwater social listening: Technical documentation, 2022.

[9] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[10] Jaimai Purev and Chimeddorj Odbayar. Mongolian nlp corpus and benchmark. In *Proceedings of Asian Language Processing*, 2019.

[11] Jaimai Purev and N Shuluu. A corpus-based mongolian grammar. In *Proceedings of the Sixth Workshop on Asian Language Resources*, 2008.

[12] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

[13] Statista. Number of internet and social media users worldwide as of january 2024, 2024.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

ХАВСРАЛТ

А. ХЭРЭГЖҮҮЛЭЛТИЙН ҮР ДҮН

...

В. КОДЫН ЖИШЭЭ

1. FastAPI Router (analysis.py) - API Endpoint хэрэгжүүлэлт:

```
from fastapi import APIRouter, File, UploadFile
from nlp_core.preprocessing import Preprocessor
from nlp_core.ner_engine import NEREngine
from nlp_core.topic_modeler import TopicModeler

router = APIRouter(prefix="/analysis", tags=["Analysis"])
preprocessor = Preprocessor()
ner_engine = NEREngine()
topic_modeler = TopicModeler()

@router.post("/upload")
async def analyze_file(file: UploadFile = File(...)):
    content = await file.read()
    text_data = content.decode("utf-8")

    texts = [line for line in text_data.split('\n') if line]
   cleaned_nlp = [preprocessor.preprocess_nlp(t) for t in texts]
    cleaned_tm  = [preprocessor.preprocess_tm(t) for t in texts]

    entities = ner_engine.recognize_batch(cleaned_nlp)
    topics   = topic_modeler.fit_transform(cleaned_tm)

    return {"status": "success",
            "entities": entities,
            "topics": topics}
```

2. Mongolian Text Preprocessing (preprocessing.py):

```
import re

def preprocess_mongolian_text(text: str) -> str:
    if not isinstance(text, str) or not text.strip():
        return ""
    text = text.lower()
    text = re.sub(r'[^\w\s]', ' ', text)
    text = re.sub(r'\d+', '', text)

    stopwords = {"байна", "болох", "тухай", "ба"}
   cleaned_words = [w for w in text.split() if w not in stopwords]

    return " ".join(cleaned_words).strip()
```